A

PROJECT REPORT

ON

GreenXchange

SUBMITTED TO
SHIVAJI UNIVERSITY, KOLHAPUR

IN THE PARTIAL FULFILLMENT OF REQUIREMENT FOR THE AWARD OF DEGREE OF BACHELOR OF ENGINEERING IN ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

SUBMITTED BY

MR. Harshad Dayanand Fagare	**23UAD015**
MR. Prathmesh Ranjit Salokhe	**23UAD060**
MR. Somesh Ramswaroop Sharma	**23UAD063**
MR. Rohit Avinash Kumbhar	**24UAD304**

UNDER THE GUIDANCE OF
Prof. P.N. Koshti

DKTE
Promoting Excellence in
Teaching, Learning & Research

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE
DKTE SOCIETY'S TEXTILE AND ENGINEERING INSTITUTE,
ICHALKARANJI

2026-2027

D.K.T.E. SOCIETY'S
TEXTILE AND ENGINEERING INSTITUTE, ICHALKARANJI
(AN AUTONOMOUS INSTITUTE)

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE

CERTIFICATE

This is to certify that, project work entitled

GreenXchange

is a bonafide record of project work carried out in this college by

MR. Harshad Dayanand Fagare	**23UAD015**
MR. Prathmesh Ranjit Salokhe	**23UAD060**
MR. Somesh Ramswaroop Sharma	**23UAD063**
MR. Rohit Avinash Kumbhar	**24UAD304**

is in the partial fulfillment of award of degree Bachelor in Engineering in Artificial Intelligence & Data Science prescribed by Shivaji University, Kolhapur for the academic year 2026-2027.

Prof. P.N. Koshti
(PROJECT GUIDE)

PROF. DR. T.I. BAGBAN
(HOD AI&DS DEPT.)

PROF. DR. L.S. ADMUTHE
(DIRECTOR)

EXAMINER: _____

DECLARATION

We hereby declare that, the project work report entitled **GreenXchange** which is being submitted to D.K.T.E. Society's Textile and Engineering Institute Ichalkaranji, affiliated to Shivaji University, Kolhapur is in partial fulfillment of degree B.E. (AI&DS). It is a bonafide report of the work carried out by us. The material contained in this report has not been submitted to any university or institution for the award of any degree. Further, we declare that we have not violated any of the provisions under Copyright and Piracy / Cyber / IPR Act amended from time to time.

MR. Harshad Dayanand Fagare	**23UAD015**
MR. Prathmesh Ranjit Salokhe	**23UAD060**
MR. Somesh Ramswaroop Sharma	**23UAD063**
MR. Rohit Avinash Kumbhar	**24UAD304**

ACKNOWLEDGEMENT

With great pleasure we wish to express our deep sense of gratitude to Prof. P.N. Koshti for his valuable guidance, support and encouragement in completion of this project report.

Also, we would like to take opportunity to thank our head of department Prof. Dr. T.I. Bagban for his co-operation in preparing this project report.

We feel gratified to record our cordial thanks to other staff members of the Department of Artificial Intelligence and Data Science for their support, help and assistance which they extended as and when required.

Thank you,

MR. Harshad Dayanand Fagare	**23UAD015**
MR. Prathmesh Ranjit Salokhe	**23UAD060**
MR. Somesh Ramswaroop Sharma	**23UAD063**
MR. Rohit Avinash Kumbhar	**24UAD304**

ABSTRACT

GreenXchange is a location-aware environmental intelligence platform that unifies data-driven plant recommendation, physical plant registration and tracking, computer-vision-based growth verification, a verified reward system, community participation management, and contextual environmental news delivery into a single cohesive ecosystem. The platform operates across web and mobile interfaces and is designed to bridge the gap between environmental awareness and accountable real-world action by creating a structured, verifiable record of individual and community contributions to urban and semi-urban greening efforts.

The system ingests real-time environmental data including air quality index values, particulate matter concentrations, weather parameters, soil characteristics, and traffic influence for any given geographic location. This data is processed into a structured environmental profile that serves as the primary input to a hybrid recommendation engine. The recommendation engine applies a two-stage pipeline consisting of a hard constraint filter that eliminates ecologically incompatible, toxic, or allergen-risk plant species based on declared user conditions, followed by a weighted scoring model that ranks the remaining candidates by pollution absorption capacity, maintenance requirement, growth rate, and ecological fit. The output is a ranked list of plant or tree species with contextual explanations and care guidance tailored to the user's specific location, space type, and personal constraints.

Once a user selects and physically plants a recommended species, the platform enables digital registration of that plant, assigning it a unique scannable identity in the form of a QR code that permanently links the physical plant to its digital record. Users submit periodic growth updates through image uploads paired with GPS coordinates. Each submission is routed through a dual-layer verification pipeline that independently performs geospatial consistency validation against the plant's registered anchor location and computer vision analysis that checks for image authenticity, plant presence, growth stage progression, and species continuity against prior verified images. Only submissions that pass both verification layers are recorded as verified growth events. Verified events trigger the reward ledger, which credits the user with points subject to time-gate eligibility rules that prevent artificial reward farming.

The platform further incorporates a community engagement module that enables users to discover, join, and contribute to local plantation drives using proximity-based geospatial queries. A personalized news feed delivers locally scoped environmental updates, air quality alerts, and drive announcements. All of these modules converge into a centralized dashboard that presents real-time environmental indicators, plant portfolio status, growth progress timelines, reward balance, nearby community drives, and curated news in a single aggregated view.

Keywords: GreenXchange; plant recommendation; computer vision verification; geospatial validation; PostGIS; reward ledger; urban greening; FastAPI; YOLO.

INDEX

Sr. No.	Chapter	Page No.
1	**Introduction** a. Introduction b. Problem Statement c. Objectives d. Scope of the project e. Timeline of the project f. Project Cost	1
2	**Literature Review** a. Literature overview b. Comparative Study of Existing Solutions c. Research Gap Identified	14
3	**Requirement Analysis** a. Requirement Gathering b. Requirement Specification c. Use case Diagram	18
4	**System Design** a. Architectural Design b. User Interface Design c. System Modeling 1. Dataflow Diagram 2. Sequence Diagram 3. Activity Diagram 4. Component Diagram 5. Deployment Diagram 6. Class Diagram 7. ER Diagram and Database Design 8. Collaboration Diagram d. Algorithmic description of each module	32
5	**Implementation** a. Environmental Setting for Running the Project b. Detailed Description of Methods c. Implementation Details	55
6	**Integration and Testing** a. Description of the Integration Modules b. Testing	67
7	**Performance Analysis**	70
8	**Future Scope**	73
9	**Applications**	75
10	**Installation Guide and User Manual**	77
11	**Publication Details**	80
12	**Industry Sponsorship Letters**	81
13	**Plagiarism Report**	82
14	**References** a. Bibliography	83

CHAPTER 1: INTRODUCTION

1.1 Introduction

Urban and semi-urban India is losing green cover faster than it is being replaced, while ambient air quality in many cities regularly crosses hazardous thresholds. Many citizens are willing to plant and care for trees, but they lack three things: guidance on which species suit their exact location, a trustworthy way to record and prove what they have planted, and a link to the community greening efforts happening around them. GreenXchange is designed to close this gap.

GreenXchange is a location-aware environmental intelligence platform that brings together data-driven plant recommendation, physical plant registration and tracking, computer-vision-based growth verification, a verified reward system, community participation management and contextual environmental news in a single web and mobile-responsive application. Its central idea is that every environmental contribution recorded on the platform must be independently verifiable, so that rewards and aggregate statistics reflect real planting activity rather than self-reported claims.

1.1.1 Approach

GreenXchange addresses the identified problem through a layered technical architecture organized around five core design principles: location specificity, ecological validity, physical verifiability, fraud resistance, and community accountability.

Location-Specific Environmental Profiling. The platform collects environmental data from authoritative third-party providers including air quality monitoring APIs, meteorological data sources, and soil classification datasets. This data is aggregated and cached per geographic tile to construct an environmental profile for any queried location. The profile captures AQI index, PM2.5 concentration, temperature range, humidity, soil pH, soil type, sunlight hours, rainfall, and urban or rural classification. Derived features including pollution severity class and climate zone are computed at ingestion time and stored with the profile to avoid recomputation at query time. All environmental data is served from cache with defined TTL values appropriate to each data source's update frequency, ensuring that recommendation inputs are current without incurring per-request external API costs.

Ecologically Valid Recommendation Engine. The recommendation pipeline operates in two sequential stages. The first stage is a hard constraint filter that eliminates candidate species based on non-negotiable criteria: temperature incompatibility, soil pH mismatch, space type incompatibility, toxicity risk for households with children or pets, and allergen risk for users

with declared sensitivities. These constraints are implemented as elimination filters, not score penalties, meaning no species that fails a hard constraint appears in the output regardless of its score on other dimensions. The second stage applies a weighted scoring function across the surviving candidates using four criteria: pollution absorption rate weighted at thirty-five percent, maintenance requirement inverse score weighted at twenty-five percent, growth rate suitability weighted at twenty percent, and ecological fit score weighted at twenty percent. Species-level data is sourced from established ecological databases including USFS, IUFRO, and ENVIS for Indian species, ensuring that absorption rates and compatibility attributes reflect measured values rather than genus-level approximations.

Physical Plant Registration and Digital Identity. Each plant selected and physically planted by a user is registered in the system through a structured registration flow that captures the plant's common name, species reference, precise GPS coordinates as the anchor location, planting date, and space type. Upon successful registration, the system generates a unique scan identifier encoded as a QR code. This identifier permanently links the physical plant to its digital record and serves as the entry point for growth update submissions, public growth record viewing, and social accountability through third-party scan access.

Dual-Layer Growth Verification. Growth update submissions are processed through two independent verification pipelines that must both pass before a submission is accepted as verified. The geospatial validation pipeline computes the Haversine distance between the submitted GPS coordinates and the plant's registered anchor location and rejects submissions that exceed a configurable tolerance radius before any image processing is initiated. The computer vision pipeline executes four sequential checks: error level analysis for image manipulation detection, plant object detection to confirm a plant is present in the frame, growth stage classification compared against the most recent prior verified image for the same plant, and species continuity verification to confirm visual consistency with the registered species. EXIF GPS metadata embedded in camera-captured images is cross-referenced against the submitted GPS coordinates as an additional authenticity signal. The two pipelines are operationally independent and both are required conditions for verification. A submission passing geo-validation but failing CV analysis, or passing CV analysis but failing geo-validation, is rejected. Verification processing is handled asynchronously via a job queue to prevent synchronous blocking of the user-facing API, with the user informed of a pending status and notified upon completion.

Verified Reward and Anti-Fraud System. The reward system credits points to users exclusively upon receipt of a verified event from the verification pipeline via an event bus. Reward credit is never initiated on submission, only on verified completion. Time-gate eligibility rules enforce minimum intervals between reward credits for the same plant to prevent artificial inflation through rapid repeated submissions. Device fingerprinting, one-scan-ID-per-account-per-reward- cycle rules, and rate limiting on registration endpoints provide additional layers of misuse resistance. All reward transactions are recorded in an append-only ledger with balance

snapshots, enabling full auditability. Community and News Infrastructure. Community drives are stored with geospatial location data and queried using PostGIS spatial indexing to surface proximity-relevant drives to each user. The news feed applies relevance scoring based on recency, category, and geographic proximity of the content's location scope to the user's location. Both modules feed into the centralized dashboard, which aggregates data from all system modules in parallel and returns partial responses with stale indicators in the event of any module timeout, ensuring dashboard availability is not dependent on full system health.

1.1.2 Expected Outcome

The primary expected outcome of GreenXchange is the establishment of a verified, auditable record of individual and community-level plant contributions at geographic scale. Unlike self-reported environmental data, each entry in the GreenXchange growth log represents an independently verified planting or growth event that has passed both geospatial and computer vision validation. The aggregate of these verified records constitutes a reliable dataset of urban greening activity that can be used by municipal authorities, environmental researchers, and community organizations to assess real-world vegetation trends, identify high-impact zones, and plan targeted interventions.

At the individual user level, the platform is expected to reduce the friction of informed plant selection by providing location-specific, ecologically valid recommendations that eliminate the need for independent research into species compatibility, toxicity, allergen risk, and pollution absorption. Users receive actionable guidance contextualized to their precise environmental conditions and personal household constraints, increasing the probability that selected plants are appropriate, sustainable, and genuinely beneficial to local air quality.

The verified reward mechanism is expected to shift the incentive structure from passive awareness to active, sustained participation. By tying rewards exclusively to verified physical actions rather than to self-reported claims, the platform creates conditions under which engagement correlates directly with genuine environmental impact. The time-gate enforcement and anti-fraud mechanisms ensure that the reward economy reflects real contribution, maintaining the integrity of the point system and the trust of legitimate participants.

The community drives module is expected to increase the discoverability and participation rates of local plantation initiatives by surfacing geographically relevant drives to users within proximity. By maintaining a structured digital record of drive participation, the platform enables organizers to quantify community engagement and provides participants with a persistent record of their collective contribution.

Over time, the accumulation of verified growth data across a user population enables longitudinal analysis of plant survival rates, growth trajectories, and species performance under specific environmental conditions. This data has potential secondary value as a training resource for improving the recommendation model through observed outcomes, moving the system from

a static rule-based and scoring approach toward a data-informed ranking model as interaction history accumulates.

The expected outcome at the system level is a self-reinforcing ecosystem in which environmental data informs better recommendations, better recommendations lead to more appropriate planting decisions, verified planting activity generates reliable environmental data, and rewards and community features sustain participation over time. The integrity of this cycle depends on the verification layer, which is the foundational technical differentiator of the platform.

1.2 Problem Statement

Problem statement: To design and develop a location-aware platform that recommends ecologically suitable, household-safe plant species for any geographic location, registers each physically planted specimen with a unique digital identity, independently verifies planting and growth through geospatial and computer-vision checks, and rewards only verified contributions, while connecting users to nearby community plantation drives and locally relevant environmental news.

1.2.1 Problem Definition

Urban environments across India and comparable high-density developing economies are experiencing a compounding environmental crisis driven by three converging forces: accelerating loss of vegetation cover, deteriorating ambient air quality, and the absence of any structured mechanism through which individual citizens can make verifiable, accountable contributions to environmental remediation at a local scale.

Air quality in major Indian metropolitan areas including Delhi, Mumbai, Pune, Bengaluru, and Kolkata routinely breaches hazardous thresholds. PM2.5 concentrations in these cities exceed World Health Organization safe limits by factors of three to twelve during peak pollution periods. Urban heat island effects intensify as green cover is displaced by concrete, raising surface temperatures, increasing energy demand, and reducing the natural cooling and filtration capacity that vegetation provides. Soil quality in urban and peri-urban zones degrades as organic matter is stripped and replaced with impermeable surfaces, reducing the viability of ground-level planting even where space nominally exists.

Despite measurable public concern about these conditions, individual participation in environmental remediation remains structurally unsupported. A person motivated to plant trees or cultivate pollution-absorbing vegetation has no reliable way to determine which specific species are ecologically appropriate for their immediate location, no platform to formally register and track what they plant, no independent mechanism to verify that their planting activity is genuine and sustained, and no connection to community-level initiatives operating near them. The motivational intent exists but the infrastructure to convert that intent into accountable, sustained,

and impactful action does not.

1.2.2 Current Challenges

Challenge 1: Absence of Location-Specific Plant Guidance

Generic gardening advice and broad environmental campaigns do not account for hyper-local variation in air quality index values, soil pH, temperature range, humidity, sunlight availability, or space constraints at the household or neighborhood level. A species that performs well as a pollution absorber in one district may be incompatible with the soil conditions two kilometers away. A plant recommended for outdoor use may carry toxicity risk unsuitable for a household with pets or children. No existing consumer-facing platform integrates real-time environmental data with species- level ecological attributes to produce recommendations that are simultaneously location-accurate, ecologically valid, and personalized to household constraints.

Challenge 2: No Verification Layer for Planting Activity

Existing environmental reward or gamification applications, where they exist, rely entirely on self-reported data. A user claims to have planted a tree and the system accepts that claim without independent verification. This design is not merely inadequate — it is actively counterproductive. It creates a direct incentive for fraudulent reporting, undermines the credibility of aggregate environmental data generated by the platform, and erodes the trust of users who contribute genuinely. Without a verification layer that independently confirms both the physical location and the visual evidence of a planting event, any reward mechanism becomes an incentive for abuse rather than impact.

Challenge 3: Fragmented and Undiscoverable Community Initiatives

Local plantation drives, municipal greening programs, and community environmental initiatives exist across most urban areas but are organized through disconnected channels including social media groups, WhatsApp broadcasts, and local notice boards. There is no centralized, location-aware platform through which a motivated individual can discover drives operating within their vicinity, register participation, and contribute to a persistent record of collective impact. The result is chronically low participation rates relative to the population willing to engage, and no durable record of community-level environmental contribution.

Challenge 4: Absence of Longitudinal Plant Tracking

Once a plant is in the ground, there is no existing mechanism through which its growth, health, and survival are monitored over time. This gap means that even when planting events occur, there is no data on whether planted species survive, how they develop under specific environmental conditions, or what the actual long-term impact of a planting initiative is. The absence of longitudinal tracking makes it impossible to build an evidence base for species performance, inform future recommendations with observed outcomes, or hold participants accountable for the sustained care of what they plant.

Challenge 5: Disconnection Between Environmental Awareness and Local Action

Environmental news and awareness content is predominantly national or global in scope. Coverage of air quality crises, deforestation statistics, and climate data rarely connects directly to conditions in a user's immediate locality or to specific actions available to that user right now. This disconnect reduces the motivational impact of awareness content and contributes to a sense of individual powerlessness in the face of systemic environmental problems.

1.2.3 Importance to Society

The societal importance of GreenXchange operates across three scales. At the individual scale, the platform directly reduces the information asymmetry that prevents motivated citizens from making ecologically sound planting decisions. By providing location-specific, constraint-aware recommendations grounded in measured species-level data, it enables individuals to take actions that produce genuine local environmental benefit rather than well-intentioned but ecologically misaligned choices. At the community scale, the platform creates the infrastructure for accountable collective action. Verified participation records, community drive management, and shared environmental dashboards transform isolated individual efforts into a coordinated, measurable, and socially visible contribution to urban greening. This visibility is itself a behavioral driver: documented collective impact sustains participation in ways that abstract environmental messaging does not. At the urban policy and research scale, the aggregate of verified planting and growth records across a user population constitutes a reliable, spatially distributed dataset of greening activity that does not currently exist in any structured form. Municipal authorities, environmental researchers, and urban planners require ground-truth data on vegetation distribution, species performance under urban conditions, and the effectiveness of community greening programs.

1.2.4 Proposed Solution Impact

GreenXchange addresses the identified challenges through a unified platform that integrates four capabilities that do not currently exist together in any available system: location-aware ecological recommendation, physical plant registration with persistent digital identity, dual-layer independent verification of planting and growth activity, and community coordination with spatial discovery.

The projected impact of this integration is measurable across multiple dimensions. Users receive plant recommendations that are calibrated to their precise environmental conditions and personal constraints, increasing the probability of successful planting and sustained plant health. Every registered planting event becomes a verifiable entry in a persistent growth log, creating accountability that self-reported systems cannot produce. The reward mechanism, tied exclusively to verified events, sustains participation over time rather than collapsing into a one-time engagement. Community drives become discoverable and their impact becomes quantifiable.

And the aggregate dataset generated by the system creates the foundation for increasingly accurate, data-informed recommendations as observed outcomes accumulate.

1.3 Objectives

1.3.1 Goals

Objective 1: Develop a Real-Time Environmental Profiling System. Build a data ingestion and caching layer that retrieves air quality, weather, and soil data from authoritative third-party sources for any queried geographic location, constructs a structured environmental profile, and serves that profile with appropriate TTL-based cache management to minimize external API dependency and ensure recommendation inputs are current and consistent.

Objective 2: Implement an Ecologically Valid, Constraint-Aware Recommendation Engine. Design and deploy a two-stage recommendation pipeline that applies hard elimination filters for ecological incompatibility, toxicity, and allergen risk before applying a weighted multi-criteria scoring function to rank surviving species. Ensure that all species-level data including absorption rates, toxicity classifications, allergen modifiers, and compatibility attributes is sourced from verified ecological datasets and stored at the species level, not the genus level.

Objective 3: Build a Plant Registration and Digital Identity System. Implement a plant registration workflow that captures species selection, precise GPS anchor location, planting date, and space type, and generates a unique server-side scan identifier encoded as a QR code that permanently links the physical plant to its digital record and enables both owner-managed updates and public growth record access.

Objective 4: Design and Deploy a Dual-Layer Growth Verification Pipeline. Build an asynchronous verification system that independently executes geospatial anchor validation and computer vision analysis on every growth update submission. The CV pipeline must perform error level analysis for manipulation detection, plant presence detection, growth stage classification against prior verified images, and species continuity checking. Both layers must pass independently before a submission is accepted as verified.

Objective 5: Implement a Fraud-Resistant Verified Reward System. Build an append-only reward ledger that credits points exclusively upon receipt of a verified event from the verification pipeline, enforces time-gate eligibility checks before crediting, and incorporates device fingerprinting, rate limiting, and scan-ID ownership binding as misuse resistance mechanisms.

Objective 6: Deliver a Location-Aware Community Engagement Module. Implement a community drives module that stores drive locations as PostGIS geometry, surfaces proximity-relevant drives to users via spatial queries, manages participation records with composite unique constraints, and maintains a participant count with both a denormalized counter for display and authoritative count derivation from the participation table.

Objective 7: Build a Personalized Environmental News Feed. Implement a news ingestion and relevance scoring system that filters and ranks content by geographic proximity to the user's location, category, and recency, delivering locally scoped environmental updates, AQI alerts, and drive announcements rather than generic global content.

Objective 8: Develop a Unified Dashboard with Partial Availability Guarantees. Build a dashboard aggregation layer that executes all module data fetches in parallel, assembles a unified response, and returns partial responses with stale indicators for any module that times out, so that the dashboard remains usable even when an individual module is slow or unavailable.

1.3.2 Expected Outcomes

The platform is expected to produce the following measurable and observable outcomes upon deployment and sustained use. Users will receive plant recommendations that are demonstrably more location-appropriate and constraint-aware than generic gardening guidance, as evidenced by the elimination of ecologically incompatible or user-unsafe species from recommendation outputs through the hard filter layer. Every planting event registered on the platform will have an independently verifiable digital record with a timestamped growth log, transforming self-reported environmental activity into an auditable contribution history. The reward system will sustain active user participation over multi-month periods by creating a direct, verifiable link between physical environmental action and platform recognition, with fraud resistance mechanisms ensuring that reward distribution reflects genuine contribution. Community drive participation rates for listed initiatives will increase relative to baseline undiscovered participation, driven by proximity-based surfacing of locally relevant drives.

The aggregate growth log will accumulate a spatially distributed, verified dataset of urban planting activity usable for secondary analysis of species performance, survival rates, and environmental impact by researchers and municipal authorities. Over time, the recommendation model will incorporate observed outcome data from the growth log to transition from a static scoring function to a data-informed ranking model, improving recommendation accuracy as the user base and interaction history grow.

1.3.3 Success Criteria

The following criteria define successful implementation and deployment of GreenXchange. The recommendation engine must return a ranked species list for any valid geographic location within three seconds, with all hard filter constraints correctly applied and verifiable against the input parameters and species dataset. Growth update submissions must pass through geo-validation before any CV processing is initiated, with out-of-bounds submissions rejected at the validation stage without consuming CV pipeline resources. The CV pipeline must complete verification for a submitted image within a defined SLA of under five minutes from queue en-

try under normal load, with verification status communicated to the user via push notification upon completion. The reward ledger must demonstrate zero instances of double credit for the same user, plant, and time-gate window combination under concurrent submission testing. The dashboard must return a usable partial response within three seconds even when one or more module data fetches are delayed or unavailable, with stale indicators correctly displayed for affected modules.

The community drive discovery query must return proximity-relevant results for a given user location using PostGIS spatial indexing within one second under normal database load. The scan ID public view must expose only district or locality-level location data for a registered plant, with precise GPS coordinates confirmed absent from any public-facing API response.

1.3.4 Target Users

Primary Users: Individual Urban and Peri-Urban Residents Adults residing in urban or peri-urban areas who have access to planting space, whether a balcony, terrace, garden, or ground-level plot, and who are motivated to contribute to local environmental improvement but lack the specialized knowledge to make informed species selections or the infrastructure to formally record and sustain their contributions.

Secondary Users: Community Organizers and Environmental NGOs Individuals and organizations that plan and execute local plantation drives, community greening programs, or school and institution-based environmental initiatives, who require a structured platform for drive creation, participant management, and impact documentation.

Tertiary Users: Municipal Authorities and Environmental Researchers Agencies and research institutions that require verified, spatially distributed data on urban planting activity, species distribution, and vegetation trends to inform policy decisions, urban planning, and academic research. These users interact primarily with aggregate data outputs rather than the individual user - facing platform.

Administrative Users: Platform Administrators Internal users responsible for reward audit, fraud investigation, species dataset management, and system health monitoring, who require access to transaction logs, verification audit trails, and user activity records.

1.4 Scope of the Project

1.4.1 What is Included

Environmental Data Integration Real-time retrieval and structured caching of AQI index, PM2.5 concentration, temperature, humidity, rainfall, soil pH, soil type, sunlight hours, and urban or rural classification from third-party environmental data APIs. Derivation of pollution severity class and climate zone as computed attributes at ingestion time. Cache management

with source- appropriate TTL values and tile-based location indexing for retrieval efficiency.

Recommendation Engine A two-stage pipeline consisting of a hard constraint filter followed by a weighted multi-criteria scoring function. Species compatibility checking against environmental profile and user-supplied constraints. Hard elimination of species based on temperature incompatibility, soil pH mismatch, space type incompatibility, toxicity risk, and allergen risk. Weighted scoring across pollution absorption capacity, maintenance requirement, growth rate suitability, and ecological fit. Return of ranked recommendations with per-species explanation strings and care guidance.

Plant Registration System User-initiated plant registration capturing species reference, GPS anchor location, planting date, space type, and common name. Server-side generation of unique scan identifier after confirmed database write. QR code generation and delivery to user. Public read-only plant profile accessible via scan ID showing growth history at locality-level geographic resolution.

Growth Update and Verification System Image and GPS submission workflow for periodic growth updates. Geospatial anchor validation using Haversine distance computation before CV processing is initiated. Asynchronous CV pipeline executing error level analysis, plant presence detection, growth stage classification against prior verified images, and species continuity checking. EXIF GPS cross-validation. Append-only growth log with timestamped verified entries. Push notification delivery of verification outcomes.

Reward Ledger System Event-driven reward crediting triggered exclusively by verified events from the verification pipeline via event bus subscription. Time-gate eligibility enforcement. Append- only reward transaction ledger with balance snapshots. Fraud resistance through device fingerprinting, rate limiting, and scan ID ownership binding. User-facing reward balance and transaction history.

Community Drives Module Drive creation by authorized organizers with geospatial location data stored as PostGIS geometry. Proximity-based drive discovery using ST_DWithin spatial queries. Participation registration with composite unique constraint preventing duplicate entries. Participant count management with both denormalized display counter and authoritative derived count.

News Feed Module Location-scoped news item ingestion with category classification. Relevance scoring based on geographic proximity, recency, and category. Personalized feed delivery filtered by user location and preferences. Coverage of local AQI events, plantation drive announcements, and environmental awareness content.

Unified Dashboard Parallel aggregation of environmental profile, plant portfolio with growth status, reward balance, nearby community drives, and personalized news feed. Partial response delivery with stale indicators on module timeout. Real-time AQI and weather panel. Plant portfolio grid with growth progress and last verified date per plant.

Security and Anti-Fraud Infrastructure JWT-based authentication. Server-side timestamp assignment on all user submissions. Image re-encoding and EXIF stripping at ingestion. GPS spoofing detection via EXIF cross-validation. Rate limiting on registration and submission endpoints. Row-level locking on reward credit operations.

1.4.2 What is Excluded

Carbon Credit or Offset Integration The platform does not compute, certify, or trade carbon offset credits. Verified planting data may form the basis for future carbon accounting integrations but this functionality is outside the current scope.

Direct Nursery or Plant Purchase Integration The platform recommends species and provides care guidance but does not integrate with e-commerce platforms, nursery inventory systems, or plant delivery services. Procurement of physical plants is entirely outside the system boundary.

Drone or Satellite Remote Sensing Environmental data is sourced from third-party ground- level and satellite monitoring APIs. The platform does not operate its own remote sensing infrastructure or process raw satellite imagery.

Municipal System Integration While the platform generates data of value to municipal authorities, direct API integration with municipal planning systems, government environmental databases, or urban development platforms is not included in the current scope.

Real-Time Species Health Monitoring via IoT Sensors The platform relies on user-submitted photo updates for growth tracking. Integration with IoT soil moisture sensors, air quality monitors, or growth sensors attached to individual plants is not included.

Multi-Language Support Beyond English The initial deployment targets English-language interfaces. Regional language localization is identified as a future scope item but is not implemented in the current version.

Native Mobile Application The current implementation targets web application and mobile- responsive web interfaces. A native Android or iOS application is identified as a future scope item.

Offline Functionality The platform requires network connectivity for all core functions including recommendation retrieval, growth update submission, and dashboard access. Offline caching or progressive web app offline modes are not included in the current scope.

1.5 Timeline of the Project

The project was planned across the academic year 2025-26 in four phases: planning, design, development and testing/delivery. Development of independent modules was overlapped wherever dependencies allowed, and the computer vision pipeline, being the highest-risk component,

was started early so that model training could run in parallel with backend development. Table 1.1 lists the planned activities and Figure 1.1 shows them as a Gantt chart.

Table 1.1: Project activity schedule

Phase	Activity	Duration	Period
Planning	Problem identification and topic finalization	3 weeks	Jul 2025
Planning	Literature survey and study of existing solutions	4 weeks	Jul - Aug 2025
Planning	Requirement gathering and SRS preparation	3 weeks	Aug - Sep 2025
Design	System architecture, DFD and UML modelling	3 weeks	Sep - Oct 2025
Design	Database schema, API contracts and UI wireframes	2 weeks	Oct 2025
Development	Environmental data service and recommendation engine	4 weeks	Oct - Nov 2025
Development	Plant registration and QR identity module	3 weeks	Nov - Dec 2025
Development	CV verification pipeline and model training	6 weeks	Nov 2025 - Jan 2026
Development	Reward ledger, community drives and news feed	4 weeks	Dec 2025 - Jan 2026
Development	Dashboard and frontend integration	5 weeks	Jan - Feb 2026
Testing	Integration and system testing	4 weeks	Feb - Mar 2026
Testing	Performance tuning and deployment	3 weeks	Mar 2026
Delivery	Report writing and documentation	5 weeks	Mar - Apr 2026
Delivery	Final review and submission	2 weeks	Apr 2026

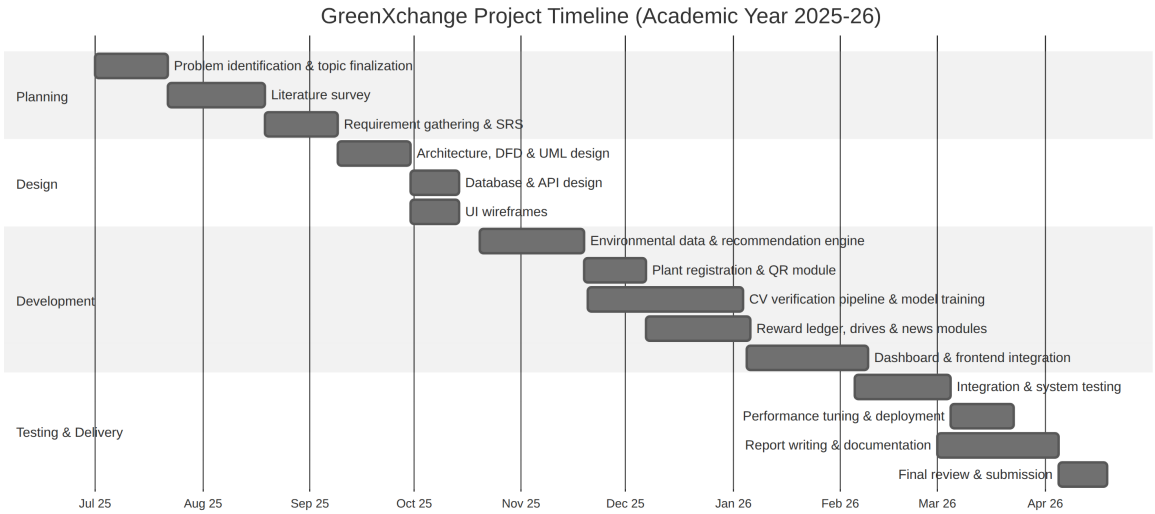


Figure 1.1: Gantt chart of the project timeline

1.6 Project Cost

GreenXchange is built entirely on open-source software (Python, FastAPI, React/Next.js, PostgreSQL/PostGIS, Redis, Celery, OpenCV, PyTorch, Docker and Nginx), so there is no licensing cost. External environmental data is obtained from the free tiers of OpenWeatherMap (up to 1,000 calls per day) and WAQI (free for non-commercial and research use), and the caching layer keeps usage well inside these limits. Development was carried out on the team members' own laptops. The main expenses are therefore cloud hosting for the test deployment, GPU time for model training and documentation. The estimated cost is summarized in Table 1.2.

Table 1.2: Estimated project cost

Sr. No.	Item	Basis	Cost (INR)
1	Development laptops	Existing student machines	0
2	Software, frameworks and libraries	Open source	0
3	Environmental data APIs (OpenWeatherMap, WAQI)	Free / research tier	0
4	Cloud VM for staging deployment (4 vCPU, 8 GB)	About Rs. 2,500 per month x 3 months	7,500
5	GPU compute for CV model training	Cloud notebook credits, 2 months	2,000
6	Domain name and SSL	1 year (SSL via Let's Encrypt)	800
7	Object storage for images	Self-hosted MinIO during development	0
8	Printing, binding and documentation	Report copies	1,500
9	Miscellaneous (internet, field visits for plant images)	Lump sum	1,200
	Total estimated project cost		**13,000**

For a production deployment serving an initial base of 1,000 to 5,000 active users, recurring infrastructure cost is estimated at USD 20 to 50 (approximately Rs. 1,700 to 4,200) per month on standard cloud providers, and lower if academic or government cloud credits are used. Because all CV inference runs on locally deployed PyTorch models rather than paid cloud vision APIs, cost grows predictably with the number of worker instances rather than with the number of images processed.

CHAPTER 2: LITERATURE REVIEW

2.1 Literature Overview

GreenXchange draws on five bodies of work: the health and environmental case for urban vegetation, recommender systems, computer vision for plant recognition and image forensics, geospatial computation, and incentive design for sustained citizen participation. This section summarizes the literature in each area and how it shaped the design of the platform.

2.1.1 Urban Air Quality and the Role of Vegetation

The World Health Organization's 2021 Global Air Quality Guidelines tightened the recommended annual mean for PM2.5 to 5 µg/m³ and the 24-hour mean to 15 µg/m³ [1]. Measured concentrations in large Indian cities are frequently several times higher, which motivates both the AQI-driven environmental profile and the pollution-severity class used by GreenXchange. The US EPA's AQI framework [2] provides the band structure (Good, Moderate, Unhealthy for Sensitive Groups, and so on) that the platform uses to bin raw AQI readings.

Nowak, Crane and Stevens [11] modelled pollution removal by urban trees and shrubs across the United States and showed that removal capacity varies substantially with canopy cover, leaf area and species characteristics. The FAO guidelines on urban and peri-urban forestry [4] similarly stress that species selection must be matched to local climate, soil and site constraints, and UNEP's work on nature-based solutions [9] frames urban greening as a practical climate adaptation measure. Together these sources justify two design choices: recommendations must use species-level rather than genus-level attributes, and the scoring function must give pollution absorption the largest weight while still respecting ecological fit.

2.1.2 Recommender Systems

Ricci, Rokach and Shapira [3] classify recommender systems into content-based, collaborative, knowledge-based and hybrid approaches. Collaborative filtering needs a large history of user-item interactions, which a new platform does not have (the cold-start problem). Knowledge-based and constraint-based recommenders, by contrast, reason directly from item attributes and explicit user requirements, and are well suited to domains where some recommendations are simply unacceptable, such as a toxic plant in a household with pets. GreenXchange therefore uses a knowledge-based hybrid: a hard constraint filter followed by a weighted multi-criteria score. As verified outcome data accumulates, learning-to-rank methods such as LambdaMART [12] provide a path to a data-driven ranking model.

2.1.3 Computer Vision for Plant Recognition

Deep convolutional networks [8], [5] have made image-based plant identification practical at scale. The Pl@ntNet project [13] and the iNaturalist species classification benchmark [14] show that large, crowd-sourced image collections can support fine-grained species recognition, although accuracy falls on rare classes and on images taken in uncontrolled conditions. One-stage object detectors, beginning with YOLO [15] and continuing in current implementations such as Ultralytics YOLOv8 [16], detect objects in a single forward pass and run fast enough for queue-based verification on modest hardware. GreenXchange uses a YOLO-family detector to confirm plant presence and a separate classifier for growth-stage labelling.

2.1.4 Image Forensics and Location Verification

Error Level Analysis (ELA), described by Krawetz [17], re-compresses a JPEG at a known quality and inspects the difference image: regions that were pasted or edited after the last save typically show a different error level from the rest of the image. ELA is not conclusive on its own, but it is a cheap first-pass signal for manipulation. For location, the Haversine formula [18] gives the great-circle distance between two latitude/longitude points and is accurate enough for the tens-of-metres tolerances needed for anchor validation. Browser and device geolocation APIs [6] provide the submitted coordinates, while EXIF metadata embedded by the camera gives an independent second reading that can be cross-checked.

2.1.5 Gamification and Verified Participation

Deterding et al. [19] define gamification as the use of game design elements in non-game contexts, and note that points and rewards sustain engagement only when users perceive them as fair and meaningful. When rewards are tied to self-reported activity, they invite gaming of the system. This observation directly motivates GreenXchange's rule that points are credited only after a verification event, together with time gates, device fingerprinting and an append-only ledger.

2.2 Comparative Study of Existing Solutions

Several existing platforms address parts of the problem. Table 2.1 compares the most relevant ones against the capabilities GreenXchange combines.

Table 2.1: Comparison of existing solutions with GreenXchange

Solution	Primary purpose	Location-specific recommendation	Verification of planting	Growth tracking	Rewards	Community drives
iNaturalist	Citizen-science species observation and identification	No	Community agrees on species ID, not on who planted what	No (single observations)	No	Projects and bioblitz events
Pl@ntNet	Image-based plant identification	No	No	No	No	No
PictureThis	Plant identification and care reminders	Generic care tips	No	Partial (care reminders)	No	No
Ecosia	Search engine funding tree-planting partners	No	Reports published by the organization	No	Tree counter only	No
Grow-Trees.com	Plant a tree on your behalf through partner NGOs	No	Organization-reported, certificate issued	No	Certificate	No
Global Forest Watch	Satellite monitoring of forest cover	No	Landscape level, not individual	Landscape level	No	No
GreenXchange	**Verified individual and community greening**	**Yes (AQI, soil, climate, household constraints)**	**Yes (geo + CV, dual layer)**	**Yes (append-only growth log)**	**Yes (verified-only, time-gated)**	**Yes (PostGIS proximity)**

The comparison shows a clear split. Identification apps (Pl@ntNet, PictureThis, iNaturalist) are strong at recognizing what a plant is but do not recommend species for a specific site or track a specific plant over time. Tree-planting programmes (Ecosia, Grow-Trees.com) plant at scale, but the individual contributor has no role in planting or verification. Satellite platforms monitor large areas but cannot attribute vegetation to individual actions. No surveyed platform connects a location-specific recommendation to a physically planted, independently verified and rewarded specimen.

2.3 Research Gap Identified

- **No site-specific, constraint-aware recommendation:** Existing consumer tools do not combine real-time AQI, climate and soil data with species-level absorption, toxicity and allergen attributes and household constraints such as pets, children and space type.
- **No verification of individual planting:** Rewards and impact counters rely on self-reported or organization-reported data. There is no independent check that the claimed

plant exists at the claimed location.

- **No longitudinal record per plant:** Once planted, specimens are not tracked, so survival and growth under specific urban conditions go unmeasured.
- **Fragmented community initiatives:** Local plantation drives are announced through disconnected channels and leave no persistent record of participation or impact.
- **Awareness disconnected from local action:** Environmental news is rarely filtered by the reader's locality or linked to actions available nearby.

GreenXchange addresses this gap by linking the full chain in a single system: location profile, then recommendation, then registration with a QR identity, then dual-layer verified growth updates, then fraud-resistant rewards, with community drives and local news around them. The dual-layer verification step is the key technical contribution, because it is what makes the platform's rewards and aggregate data trustworthy.

CHAPTER 3: REQUIREMENT ANALYSIS

3.1 Requirement Gathering

Requirements were gathered from the four user groups identified in Section 1.3.4 using a combination of techniques, so that functional needs, domain constraints and anti-fraud requirements were all captured before design began.

Table 3.1: Requirement gathering techniques and findings

Technique	Participants / sources	Key findings
Questionnaire survey	Students, faculty and residents of Ichalkaranji and Kolhapur (online form)	Most respondents were willing to plant but did not know which species suit their balcony or terrace; recognition and reminders were cited as motivators.
Informal interviews	Local plantation drive volunteers and a nursery owner	Drives are announced over WhatsApp and social media; there is no record of who attended or whether saplings survived. Survival after the first summer is the main problem.
Domain study	WHO/EPA AQI guidance, FAO urban forestry guidelines, species databases (USFS, IUFRO, ENVIS)	Species must be matched on temperature range, soil pH and space type; toxicity and allergen attributes must be stored per species.
Study of existing apps	iNaturalist, Pl@ntNet, PictureThis, tree-planting platforms	Identification is mature, but no app recommends for a site, tracks a specific plant, or verifies planting (Chapter 2).
Brainstorming and misuse-case analysis	Project team and guide	Likely abuse: re-uploading old or downloaded photos, spoofed GPS, rapid repeated submissions, multiple accounts. Each led to a specific countermeasure.

The findings were consolidated into a Software Requirements Specification of eight functional requirements (FR1 to FR8) and a set of non-functional requirements covering performance, security, usability, reliability, scalability and maintainability, which were reviewed with the project guide before the design phase.

3.2 Requirement Specification

3.2.1 Functional Requirements

FR1: Real-Time Environmental Profiling

Description The system must retrieve, process, and cache environmental data for any geographic location queried by the user or triggered by a recommendation request. The environmental profile must include air quality index, PM2.5 concentration, temperature, humidity, soil pH, soil type, sunlight hours, rainfall, and urban or rural classification. Two derived attributes, pollution severity class and climate zone, must be computed from raw inputs at ingestion time and stored with the profile. The profile must be served from cache on subsequent requests until the TTL for the relevant data fields expires, at which point a fresh retrieval from external providers must be triggered asynchronously. The system must never call external environmental APIs synchronously per user request.

Input Geographic coordinates as latitude and longitude decimal values submitted by the user at onboarding or derived from device location permission. Location tile identifier computed from coordinates for cache keying.

Output A structured environmental profile object containing all raw and derived attributes with a cached-at timestamp and TTL value. Staleness indicator flag set to true if the cached profile is expired and a refresh has been triggered but not yet completed.

Priority Critical. This module is a prerequisite for the recommendation engine. Without a valid environmental profile, no recommendation can be generated.

FR2: Constraint-Aware Plant Recommendation Engine

Description The system must produce a ranked list of plant or tree species suitable for the user's location and personal constraints. The recommendation pipeline must execute in two strictly sequential stages. Stage one is a hard constraint filter that eliminates species from the candidate set based on ecological incompatibility with the environmental profile and user-declared constraints. Eliminated species must not appear in the output under any condition, regardless of their score on other dimensions. Stage two applies a weighted multi-criteria scoring function to the surviving species and ranks them in descending order of composite score.

Input Environmental profile for user location. User-supplied filter parameters: space type as indoor or outdoor, available space in square meters, experience level, declared allergies as an array of strings, presence of pets as boolean, presence of children as boolean.

Output Ordered list of recommended plant species with composite score, rank position, explanation string, and care guidance. If the hard filter eliminates all candidates, an empty list with reason codes specifying which constraint eliminated all species must be returned. No fallback to soft filtering is permitted.

Priority Critical. This is a primary value-delivery function of the platform and is the entry point for new user activation.

FR3: Plant Registration and Scan Identity Assignment

Description The system must allow authenticated users to register a physical plant by providing its species reference, a common name, precise GPS coordinates, planting date, and space type. Upon successful database write of the plant record, the system must generate a unique scan identifier server-side using a base62-encoded fragment of the plant UUID. This identifier must be encoded into a QR code and delivered to the user for physical labeling of the registered plant. The scan identifier must be permanently and exclusively bound to the registering user's account for ownership and reward purposes. Any authenticated user scanning the QR code must receive a read-only public view of the plant's growth record showing locality-level location data only. Precise GPS coordinates must never appear in any public-facing response.

Input Authenticated user session. Plant registration payload: species ID, common name, GPS latitude and longitude, planting date, space type.

Output Plant record created in database with unique plant ID and scan ID. QR code image delivered to user. Dashboard updated to include newly registered plant in portfolio. Confirmation response with plant ID and scan ID.

Priority Critical. This function establishes the physical-digital link that all subsequent growth tracking and reward activity depends on.

FR4: Growth Update Submission and Dual-Layer Verification

Description The system must allow registered plant owners to submit periodic growth updates consisting of an image and GPS coordinates associated with a specific registered plant identified by its plant ID or scan ID. Upon submission, the system must immediately execute geospatial anchor validation by computing the Haversine distance between the submitted GPS coordinates and the plant's registered anchor location. If the distance exceeds the configured tolerance radius, the submission must be rejected before any image processing is initiated and a LOCATION - MISMATCH error must be returned. If geo-validation passes, the system must write a pending growth update record to the database and enqueue the image to the asynchronous CV verification worker queue. The CV worker must execute four sequential checks: error level analysis for manipulation detection, plant presence detection using a trained object detection model, growth stage classification compared against the most recent prior verified image for the same plant, and species continuity checking against the registered species. If a prior verified image does not exist for the plant, growth stage comparison must be skipped and the submission treated as a baseline image. EXIF GPS metadata embedded in the image must be extracted and cross-referenced against the submitted GPS. A delta exceeding two hundred meters must flag the submission for manual review rather than auto-verification. Upon completion, the verification

worker must update the growth update record with verification status, confidence score, growth stage label, model version, and rejection reason if applicable. A verification completed event must be emitted to the event bus regardless of pass or fail outcome.

Input Authenticated user session. Growth update payload: plant ID, image binary, submitted GPS latitude and longitude. Optionally EXIF GPS embedded in image metadata.

Output Pending growth update record written to database. Asynchronous verification job enqueued. Immediate API response with update ID and status pending. Push notification delivered to user on verification completion with status verified or rejected and rejection reason if applicable. Verified update appended to plant's growth log.

Priority Critical. This function is the core accountability mechanism of the platform and the prerequisite for reward crediting.

FR5: Verified Reward Ledger and Time-Gate Enforcement

Description The system must credit reward points to a user's account exclusively when a verification completed event with status verified is received from the event bus. Reward crediting must never be initiated on submission or on pending status. Before crediting, the reward service must query the reward transactions table to retrieve the timestamp of the most recent reward credited to the same user for the same plant. If the elapsed time since the last reward is less than the configured minimum reward interval for the applicable event type, the credit must be withheld and the eligibility check result must be logged. If the time gate passes, the reward service must compute the applicable points for the trigger event type, compute the user's current balance by reading the balance snapshot of the most recent transaction, insert a new reward transaction record with the points, trigger event, and updated balance snapshot, and publish a reward credited event to the event bus. Row-level locking must be applied during the eligibility check and insert to prevent concurrent double-credit for the same user and plant combination. The reward transactions table must be append-only with no update or delete operations permitted under any operational condition.

Input Verification completed event from event bus containing update ID, user ID, plant ID, and verification status.

Output New reward transaction record inserted in ledger with points, trigger event, and balance snapshot. Reward credited event published to event bus. Push notification delivered to user with points earned and updated balance. If time gate fails, no transaction is inserted and an eligibility failure log entry is written.

Priority Critical. This function directly determines platform trust. Premature, incorrect, or duplicated reward crediting is both a UX failure and a system integrity failure.

FR6: Community Drive Management and Proximity Discovery

Description The system must allow authorized organizers to create community plantation drives by providing a title, description, geographic center point, radius in meters or an optional geofence polygon, start date, and end date. Created drives must be stored with PostGIS geometry types for spatial querying. Authenticated users must be able to discover drives near their current location through a proximity query using ST_DWithin against the drive location center and the user's coordinates. Users must be able to join a drive through a single action that creates a participation record. The system must enforce a composite unique constraint at the database level preventing a user from joining the same drive more than once. The drive's participant count must be incremented on each participation record insert. Users must be able to view drive details including title, description, location, date range, and current participant count.

Input For creation: organizer authenticated session, drive payload with title, description, location center coordinates, radius or polygon geometry, start date, end date. For discovery: user location coordinates. For join: user authenticated session, drive ID.

Output Drive record created with spatial geometry stored. Proximity-ordered drive list returned for discovery. Participation record created with duplicate prevention enforced. Participant count incremented. Drive details response assembled.

Priority High. This function supports the community engagement objective and contributes to long-term user retention.

FR7: Personalized Location-Scoped News Feed

Description The system must deliver a curated news feed to each user consisting of items filtered and ranked by geographic proximity to the user's location, content category, and recency. News items must be categorized as local AQI alerts, plantation drive announcements, or environmental awareness content. Items with a defined location scope must be ranked higher for users whose location falls within proximity of the item's scope center. Items without a location scope must be treated as globally relevant and included for all users. The feed must not deliver generic national or global environmental content without applying proximity and relevance filtering. Relevance scores must be precomputed or computed at feed assembly time and used as the primary sort key.

Input Authenticated user session with location coordinates. Optional category filter parameter.

Output Ordered list of news feed items with title, content summary, category, source URL, tags, and relevance score. Items ordered by descending relevance score with recency as a tiebreaker.

Priority Medium. This function supports user retention and environmental awareness but does not block core platform functionality.

FR8: Unified Dashboard with Partial Availability

Description The system must provide a single dashboard endpoint that aggregates data from all active system modules in parallel and returns a unified response within a defined latency target. The dashboard must include the following panels: environmental profile panel showing current AQI, PM2.5, temperature, and weather conditions for the user's location; plant portfolio panel showing all registered plants with latest growth stage, last verified date, and accumulated points per plant.

Input Authenticated user session with location coordinates.

Output Unified dashboard response object with data from all five panels. Stale indicator flags per panel where applicable. Response assembled within defined latency target under normal load.

Priority High. The dashboard is the primary user touchpoint and the surface through which all module functionality is perceived and trusted by the user.

Note on FR8: The dashboard also includes a rewards panel (current balance and recent transactions), a community panel (nearby drives with participant counts) and a news panel (top personalized items), giving five panels in total.

Table 3.2: Summary of functional requirements

ID	Requirement	Priority
FR1	Real-time environmental profiling	Critical
FR2	Constraint-aware plant recommendation	Critical
FR3	Plant registration and scan identity	Critical
FR4	Growth update and dual-layer verification	Critical
FR5	Verified reward ledger and time gate	Critical
FR6	Community drives and proximity discovery	High
FR7	Personalized location-scoped news feed	Medium
FR8	Unified dashboard with partial availability	High

3.2.2 Non-Functional Requirements

Performance:

Response time

The recommendation endpoint must return a complete ranked species list within three seconds for any valid geographic location under normal load. This includes environmental profile retrieval from cache, hard filter execution across the full species dataset, weighted scoring computation, and response assembly. Cache miss scenarios where an environmental profile must be freshly retrieved from external APIs are exempt from this SLA and must display a loading state

to the user during retrieval. The growth update submission endpoint must return an acknowledgment response with pending status within two seconds of receiving the image upload. The two-second target covers image receipt, EXIF extraction, geo-validation computation, pending record write, and job queue enqueue. It explicitly excludes CV verification processing time, which is asynchronous.

The CV verification pipeline must complete processing for a queued job within five minutes under normal worker load. This is the end-to-end time from job enqueue to verification status update and event emission. This SLA must be monitored per job and escalated if the ninety-fifth percentile exceeds five minutes under production load. The dashboard aggregation endpoint must return a usable response within three seconds under normal load. All module data fetches execute in parallel. Individual module fetch timeout threshold is set at two seconds, after which the module returns a stale indicator and the dashboard assembles with available data. The community drive discovery query must return proximity-ordered results within one second under normal database load, relying on the GiST spatial index on the location center column.

Throughput

The backend API must sustain a minimum of two hundred concurrent requests per second across all endpoints under sustained load without degradation of response time SLAs. The CV verification worker pool must process a minimum of fifty verification jobs per hour per worker instance, with horizontal scaling available by adding worker containers without code changes. The database must sustain a minimum of five hundred read queries per second and one hundred write queries per second under peak load using connection pooling with a minimum pool size of twenty connections per application server instance.

Security:

Authentication

All API endpoints except the public plant profile view via scan ID and the news feed unauthenticated preview must require a valid JWT access token in the Authorization header. Tokens must be signed using RS256 asymmetric signing to allow token verification without distributing the signing key. Access tokens must have a maximum lifetime of fifteen minutes. Refresh tokens must have a maximum lifetime of seven days and must be rotated on each use. Failed authentication attempts must be rate-limited at five consecutive failures per IP address within ten minutes, after which the IP must be temporarily blocked for thirty minutes. Token validation must occur at the API gateway layer before any request reaches an application service.

All administrative endpoints must require an additional role claim in the JWT payload specifying admin scope. Role validation must be enforced at the service layer, not only at the gateway, to prevent privilege escalation through gateway bypass.

Encryption

All data in transit must be encrypted using TLS 1.2 minimum, with TLS 1.3 preferred.

Self- signed certificates are not permitted in production. All data at rest in the primary PostgreSQL database must be encrypted at the storage volume level. Object storage containing uploaded and re- encoded growth images must enforce server-side encryption using AES-256. Password hashes must use bcrypt with a minimum work factor of twelve or argon2id. MD5, SHA1, and unsalted hash functions are prohibited. Database connection strings, API keys for external environmental data providers, and JWT signing keys must be stored in environment variables or a secrets management system and must never appear in source code, configuration files committed to version control, or application logs.

All uploaded images must be re-encoded server-side before storage to strip EXIF metadata beyond the GPS fields required for cross-validation, eliminating the risk of embedded executable payloads in image metadata. The original client-uploaded binary must be discarded after re-encoding and must not be persisted.

Usability:

UI/UX

The dashboard must present all five panels in a single viewport on desktop without requiring scroll to access any primary information panel. On mobile viewports the panels must stack vertically with the environmental panel at the top, plant portfolio below, and community drives and news feed accessible through tabbed navigation to preserve usability on small screens. The growth update submission flow must complete in five or fewer user interactions from QR scan to submission confirmation. The flow is: scan QR code, open update form, capture image through in-app camera, confirm GPS auto-population, submit. Gallery import as the sole image source must not be permitted as the primary flow because gallery images frequently lack EXIF GPS metadata, which is required for cross-validation.

Verification pending state must be displayed as an explicit labeled status indicator on the growth update entry, distinct from verified and rejected states. The user must not interpret a pending state as a completed verification. Push notification on verification outcome must be delivered within one minute of the verification worker completing its processing. The recommendation output must present each species recommendation with a visual card showing the plant image, common name, scientific name, pollution absorption rating as a visual indicator, maintenance level, and an explanation string of no more than two sentences describing why this species suits the user's specific conditions. The explanation string must reference actual values from the user's environmental profile, not generic species descriptions.

All error states including LOCATION-MISMATCH on growth update, empty recommendation list with reason codes, and failed verification with rejection reason must be presented to the user in plain language that describes the problem and the available corrective action. Technical error codes must not be exposed in user-facing messages.

Others:

Reliability

The platform must maintain a minimum uptime of ninety-nine point five percent for all user- facing API endpoints measured on a rolling thirty-day basis. Planned maintenance windows must not exceed two hours per month and must be scheduled during off-peak hours. The CV verification worker must implement automatic retry with exponential backoff on transient processing failures, with a maximum of three retry attempts before a job is moved to a dead letter queue for manual review. No growth update must be silently discarded. Every submission that passes geo-validation must either be verified, rejected with a reason, or moved to manual review with the user notified of the delay.

The reward ledger must guarantee zero data loss. Reward transaction records must be written to the primary database before the reward credited event is published to the event bus. If the database write fails, the event must not be published and the operation must be retried. The ledger must never enter a state where a reward credited notification has been delivered to the user but no corresponding transaction record exists in the database. Database backups must be taken daily with point-in-time recovery capability for the primary PostgreSQL instance. Backup retention must be a minimum of thirty days. Restoration from backup must be tested quarterly.

Scalability

The architecture must support horizontal scaling of the FastAPI application server layer by adding instances behind the Nginx reverse proxy without changes to application code or configuration beyond load balancer registration. Statelessness of the application layer is enforced by the JWT authentication model and the Redis-backed session and cache layer, ensuring no application instance holds state that prevents it from being replaced or replicated. The Celery worker pool for CV verification must scale horizontally by adding worker containers. Worker count must be configurable through environment variables without requiring application redeployment. Worker autoscaling based on queue depth must be implementable using Celery's built-in autoscaler or a container orchestration scaling policy.

The PostgreSQL primary database must be provisioned with read replica capability. Read-heavy queries including dashboard aggregation, news feed assembly, and recommendation engine species queries must be routable to read replicas to distribute read load without contacting the primary. Write operations including growth update inserts, reward transaction inserts, and plant registration must always contact the primary. The object storage layer must handle image storage at scale without capacity planning involvement, using an S3-compatible API that abstracts underlying storage infrastructure.

Maintainability

All backend services must be structured as independent modules with clearly defined interfaces, allowing individual modules to be modified, tested, or replaced without cascading changes to other modules. The recommendation engine, CV pipeline, reward service, and com-

munity module must each be independently deployable as separate Celery task sets or FastAPI routers.

All database schema changes must be managed through a versioned migration system. No schema changes may be applied directly to the production database without a corresponding migration script that is version-controlled, reviewed, and tested in a staging environment. The plant species dataset must be updatable through migration scripts without requiring application code changes. Species attributes, absorption rates, and compatibility flags must be data-driven, not hardcoded.

All API endpoints must be documented through auto-generated OpenAPI specifications exposed at a documentation endpoint. Documentation must be kept current automatically by FastAPI's schema generation from Pydantic model definitions, requiring no manual documentation maintenance. Application and worker logs must be structured as JSON with consistent field naming including timestamp, service name, endpoint or task name, request ID, user ID where applicable, and error details. Structured logging enables log aggregation and query without log parsing.

3.2.3 Feasibility Analysis

Technical Feasibility

GreenXchange is technically feasible within the constraints of current open-source software capabilities and standard cloud or self-hosted deployment infrastructure. Each component of the system maps directly to a mature, production-validated technology with documented deployment patterns.

The environmental data ingestion layer depends on third-party APIs from OpenWeatherMap and WAQI, both of which offer free tiers sufficient for development and early deployment and paid tiers with defined rate limits and SLAs for production scale. The data returned by these APIs is structured and well-documented, and integration is straightforward through standard HTTP client libraries with Redis-backed caching managed by Celery periodic tasks.

The recommendation engine's hard filter and weighted scoring pipeline involves no machine learning model training in the initial phase. It is a deterministic algorithm operating on a structured species dataset. The computational complexity of filtering and scoring across a species dataset of realistic size, estimated at five hundred to two thousand species, is negligible and well within the response time SLA. The transition to a trained ranking model in future phases requires sufficient interaction log accumulation, which is a data availability question, not a technical barrier.

The computer vision verification pipeline represents the highest technical complexity in the system. Plant object detection using YOLO-architecture models fine-tuned on plant imagery is a documented and achievable task with publicly available pre-trained weights as a start-

ing point. Error level analysis for manipulation detection is implemented by established open-source tools. Growth stage classification against prior images is the most novel component and requires a purpose-built training dataset with botanically valid growth stage labels. This is the one component where technical risk is non-trivial: model accuracy for growth stage classification will depend on the quality and size of the training dataset, and initial model performance may require iteration. This risk is manageable because the system's architecture supports model version recording on every verification, enabling re-verification of historical submissions when the model improves, and because the manual review fallback path provides a human-supervised alternative for edge cases.

PostGIS spatial queries for geo-validation and drive discovery are standard database operations with well-documented performance characteristics. The Haversine distance computation for geo- validation is a simple arithmetic operation. ST_DWithin proximity queries with GiST indexes are a routine PostgreSQL deployment pattern. No novel geospatial processing is required. The asynchronous verification architecture using Celery and Redis is a documented, widely deployed pattern for computationally intensive background processing. The event bus pattern for reward decoupling from the CV pipeline is implementable using Redis Pub/Sub or Celery's task chaining capabilities without requiring a dedicated message broker infrastructure beyond the existing Redis instance.

Economic Feasibility

GreenXchange is economically feasible for development and initial deployment using a combination of open-source software, free-tier external APIs, and low-cost cloud infrastructure. Development cost is primarily labor. The technology stack consists entirely of open- source components with no licensing cost: Python, FastAPI, React, PostgreSQL, PostGIS, Redis, Celery, OpenCV, PyTorch, Docker, and Nginx are all freely available. Object storage can be self-hosted using MinIO during development and early deployment, deferring AWS S3 costs until scale justifies the migration. External API costs during development and initial deployment are manageable within free tiers. OpenWeatherMap's free tier provides up to one thousand API calls per day, sufficient for development and early user testing. The WAQI API provides free access for non- commercial and research use. Both providers offer paid plans with defined per-call pricing for production scale, and the caching architecture designed into the system minimizes API call volume by serving environmental profiles from cache with TTL management, reducing external API calls to a small fraction of total user requests.

Infrastructure cost for a production deployment handling an initial user base of one thousand to five thousand active users is estimated as follows. A single cloud virtual machine instance with four virtual CPUs and eight gigabytes of RAM is sufficient to host the FastAPI application server, Celery workers, and Nginx. A managed PostgreSQL instance or a self-hosted PostgreSQL container with PostGIS on a dedicated volume handles database requirements. A

Redis instance handles caching and task queuing. Object storage for images accumulates at a rate proportional to growth update volume. At the estimated submission rates for an initial deployment, total infrastructure cost is within a range of twenty to fifty US dollars per month on standard cloud providers, or lower on academic or government cloud credits commonly available to student projects.CV model inference cost is contained by the asynchronous architecture. Workers process images from a queue and can be scaled to match demand without incurring per-inference API costs, because all CV processing uses locally deployed PyTorch models rather than paid cloud vision APIs. Economic Feasibility Verdict: Feasible within student project constraints for development and initial deployment. Costs scale predictably with user growth and are deferrable to paid tiers only at meaningful user scale.

Operational Feasibility

GreenXchange is operationally feasible as a student or small-team developed and maintained platform given the architecture choices made to minimize operational complexity. Docker Compose orchestration means the full system can be brought up or down with a single command in development and early production, without requiring Kubernetes or container orchestration expertise. This reduces deployment and maintenance complexity to a level manageable by a small team. The modular service architecture means that operational issues in one module, such as a CV worker outage, do not cascade to user-facing failures across the full platform. The dashboard's partial availability design means users continue to receive core functionality even during module-level incidents. This significantly reduces the operational severity of partial outages.

The append-only design of the growth log and reward ledger eliminates the risk of accidental data corruption through update or delete operations on critical records. Operational mistakes on these tables are constrained to insert-level errors, which are easier to detect and correct. Structured JSON logging across all services enables operational monitoring without requiring a dedicated log management platform in early deployment. Logs can be inspected directly from container output or forwarded to a free-tier log aggregation service. The species dataset being data-driven and manageable through migration scripts means that the ecologically critical input data for the recommendation engine can be updated, corrected, or expanded by someone with database access and migration tooling, without requiring code changes or redeployment.

The primary operational risk is CV model maintenance. As the platform accumulates more diverse growth images, the initial model may encounter edge cases that require retraining. The system's model version recording and re-verification capability mitigates this by allowing historical verifications to be revisited when a better model is available, but model retraining itself requires ML engineering effort that must be factored into long-term operational planning. Operational Feasibility Verdict: Feasible for small-team operation with the described architecture. CV model maintenance represents the primary ongoing operational investment beyond standard infrastructure management.

3.3 Use Case Diagram

Figure 3.1 shows the actors of GreenXchange and the use cases available to each. The Citizen User is the primary actor. The Drive Organizer creates and manages plantation drives, any person scanning a plant's QR code acts as a Public Scanner with read-only access, and the Administrator handles manual reviews, species data and audits. The Environmental Data Provider and CV Verification Engine are secondary (system) actors that support the profiling and verification use cases.

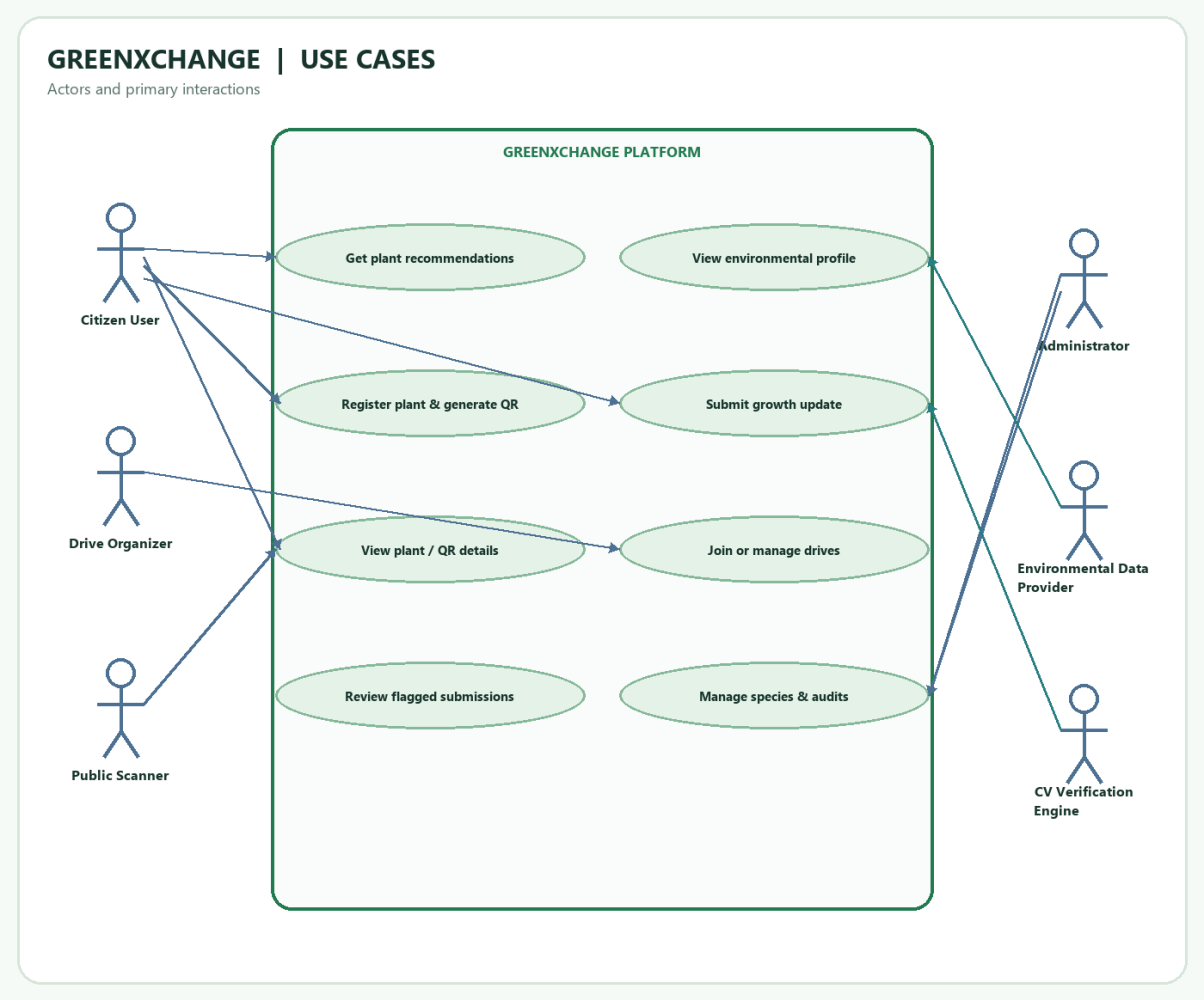


Figure 3.1: Use case diagram of GreenXchange

Table 3.3: Use case descriptions

Use case	Actor	Description
Get Plant Recommendations	Citizen User	Enter space type, area, experience, allergies and pets; receive a ranked, explained species list (includes View Environmental Profile).
Register Plant & Generate QR	Citizen User	Register species, GPS anchor, planting date and space type; receive a scan ID and QR label.

Table 3.3: Use case descriptions

Use case	Actor	Description
Submit Growth Update	Citizen User	Scan QR, capture photo in-app and submit with GPS; verified asynchronously by the CV engine.
View Rewards & History	Citizen User	See point balance and the append-only transaction history.
Discover & Join Drives	Citizen User	List drives near the user's location and join with one action.
Create / Manage Drive	Drive Organizer	Create a drive with location, radius or polygon and dates; view participants.
View Public Plant Profile	Public Scanner	Read-only growth history at locality-level location only.
Manual Review of Flagged Updates	Administrator	Resolve submissions flagged for EXIF/GPS mismatch or low CV confidence.
Manage Species Dataset	Administrator	Add or correct species attributes through versioned migrations.
Audit Rewards & Fraud Logs	Administrator	Inspect reward ledger, eligibility failures and device fingerprints.

CHAPTER 4: SYSTEM DESIGN

4.1 Architectural Design

GreenXchange follows a layered, service-oriented architecture (Figure 4.1). Each layer has one responsibility and communicates with neighbouring layers through well-defined interfaces, which allows the computationally heavy verification pipeline to scale independently of the user-facing API.

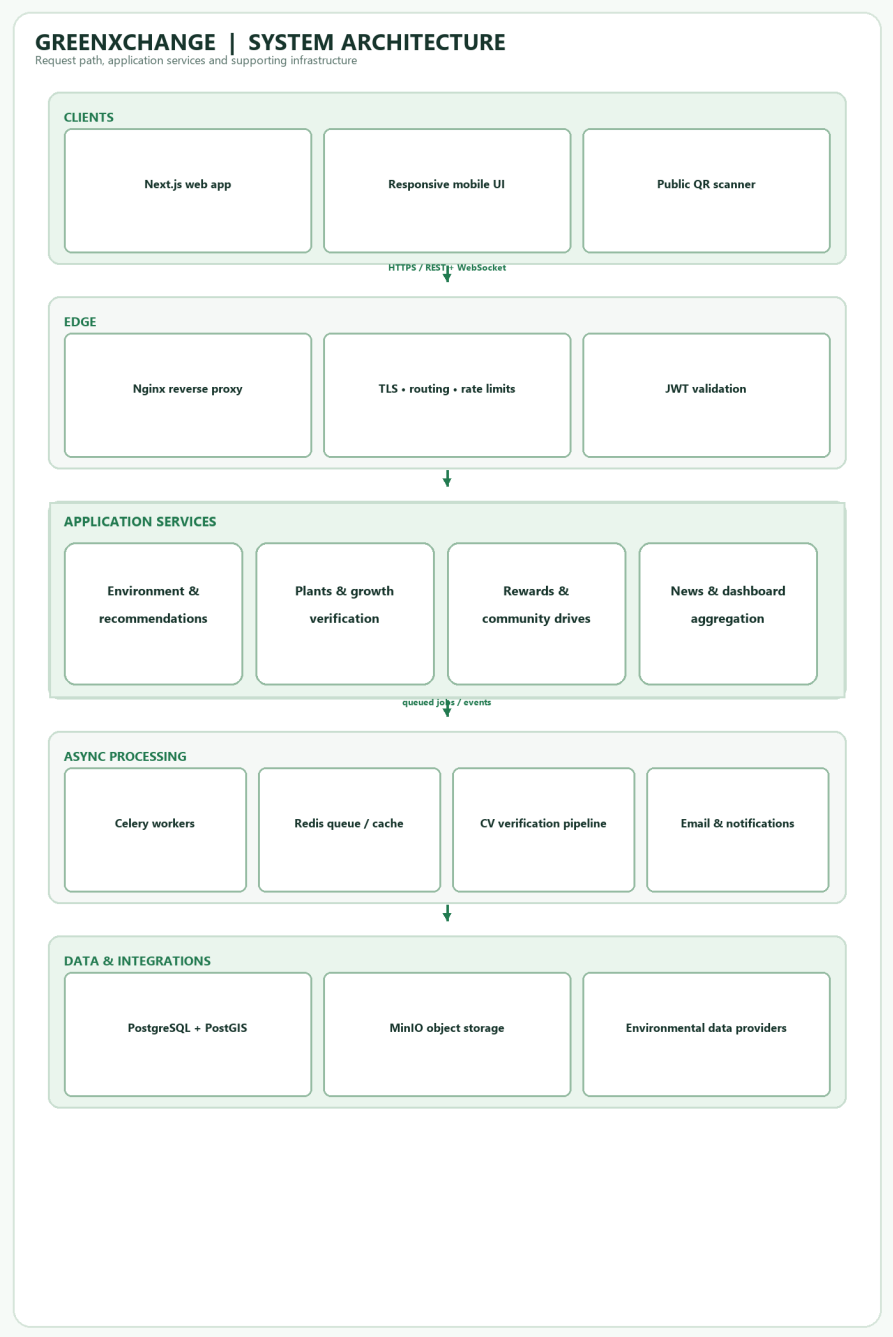


Figure 4.1: GreenXchange system architecture

- **Client layer:** The web application, the mobile-responsive interface and the QR scanner interface communicate with the backend over HTTPS REST, with WebSocket updates for the live dashboard.
- **API gateway layer:** Performs routing, rate limiting and JWT validation before any request reaches an application service.
- **Application services layer:** Seven services: Environmental Data, Recommendation Engine, Plant Registration, Growth Verification, Reward Ledger, Community and Drives, and News Feed, plus a Dashboard Aggregator.

- **Async processing layer:** A Redis-backed job queue feeds the Computer Vision worker and Geo-Validation worker, which emit verification results as events.
- **Event bus layer:** Plant-registered, verification-completed and reward-credited events decouple producers from consumers, so the reward logic never runs inside the verification code path.
- **Data layer:** PostgreSQL with PostGIS (relational and spatial data), S3-compatible image storage, a Redis cache with TTL, an append-only growth log and an external API cache store.
- **External integrations layer:** AQI and weather providers (and, optionally, government environmental data APIs), always accessed through the cache rather than per user request.
- **Cross-cutting concerns:** JWT authentication, rate limiting, input validation, image re-encoding at ingestion, EXIF stripping and server-side timestamp assignment are applied across all layers.

4.1.1 Design Decisions and Justification

Decision 1: Asynchronous CV Verification via Job Queue

Decision: Use an asynchronous job queue to handle computer vision (CV) verification of growth updates.

Rationale: CV processing is computationally intensive and can introduce high latency if executed synchronously within request-response cycles. By offloading verification tasks to a job queue (e.g., Redis-backed workers), the system maintains responsiveness for user-facing APIs while enabling scalable background processing. This approach also supports retry mechanisms, fault isolation, and horizontal scaling of worker nodes independently from API servers.

Trade-off: Users do not receive immediate verification results, introducing eventual consistency. Additional infrastructure complexity is required to manage queues and workers.

Decision 2: Hard Filter Constraint Layer in Recommendation Engine

Decision: Apply strict constraint-based filtering before scoring candidate plant species.

Rationale: Hard filters ensure that only environmentally compatible species (temperature, AQI tolerance, rainfall) enter the recommendation pipeline, preventing irrelevant or harmful suggestions. This reduces computational overhead in downstream ranking and improves recommendation quality by eliminating infeasible options early. It also simplifies explainability since excluded species clearly violate constraints.

Trade-off: Potentially suitable edge-case species may be excluded due to rigid thresholds. Flexibility is reduced compared to soft scoring approaches.

Decision 3: Server-Side Timestamp Assignment for All User Submissions

Decision: Assign timestamps on the server rather than trusting client-provided timestamps.

Rationale: Client clocks are unreliable and susceptible to manipulation, leading to inconsistencies in temporal ordering and audit trails. Server-side timestamps ensure uniformity, integrity, and accurate sequencing of events across distributed systems.

Trade-off: Slight loss of client-side temporal context (e.g., offline capture time). Requires additional handling if true capture time must be preserved separately.

Decision 4: PostGIS Extension for Spatial Queries

Decision: Use PostGIS to enable geospatial indexing and querying.

Rationale: GreenXchange relies heavily on location-based data such as plant growth, environmental matching, and locality-based recommendations. PostGIS provides optimized spatial indexing (GiST), geometric operations, and efficient proximity queries directly within PostgreSQL. This avoids the need for external geospatial services and keeps data processing close to storage.

Trade-off: Increased database complexity and operational overhead. Requires specialized knowledge for optimization and query tuning.

Decision 5: Event Bus for Reward Trigger Decoupling

Decision: Use an event bus to decouple reward logic from core application workflows.

Rationale: Publishing events (e.g., "Growth Verified") allows independent services to consume and trigger rewards without tightly coupling business logic. This enhances extensibility, enabling new reward mechanisms without modifying existing services. It also improves system resilience by isolating failures in reward processing.

Trade-off: Eventual consistency in reward delivery and increased debugging complexity due to asynchronous flows. Requires event schema governance.

Decision 6: Append-Only Growth Log Table

Decision: Maintain an append-only table for tracking plant growth updates.

Rationale: An append-only model preserves the full historical record of growth events, enabling auditability, time-series analysis, and rollback-free consistency. It aligns well with event sourcing principles and avoids data loss from overwrites.

Trade-off: Storage usage grows continuously and requires retention strategies. Querying current state may require aggregation over multiple records.

Decision 7: EXIF GPS Cross-Validation Against Submitted GPS

Decision: Validate user-submitted GPS coordinates against EXIF metadata from uploaded images.

Rationale: This mechanism reduces fraud by ensuring that uploaded images correspond to the claimed location. EXIF metadata provides an independent data source that can be cross-checked for consistency.

Trade-off: EXIF data may be missing or stripped, leading to false negatives. Additional processing overhead is introduced for metadata extraction.

Decision 8: Cached Environmental Profiles with TTL Instead of Live API Calls Per Request

Decision: Cache environmental data with a time-to-live (TTL) instead of fetching it for every request.

Rationale: External environmental APIs introduce latency, cost, and potential rate limits. Caching reduces repeated calls for the same location, improving response time and system efficiency.

Trade-off: Data may become slightly stale within the TTL window. Cache invalidation and synchronization add complexity.

Decision 9: Species-Level Plant Data Instead of Genus-Level Aggregation

Decision: Store and operate on plant data at the species level.

Rationale: Species-level granularity enables more precise recommendations, as environmental tolerances and growth characteristics can vary significantly within a genus. This improves personalization and ecological accuracy. It also supports richer analytics and better user trust in recommendations.

Trade-off: Increased data volume and complexity in maintaining detailed species datasets. Requires more comprehensive data sourcing and validation.

Decision 10: Public Scan ID Showing Locality Only, Not Precise GPS Coordinates

Decision: Expose only generalized locality information (e.g., city or area) instead of exact GPS coordinates.

Rationale: Protecting user privacy is critical, especially when dealing with geolocation data. Limiting precision prevents misuse such as stalking or unauthorized tracking. It balances transparency (for community visibility) with safety and compliance considerations.

Trade-off: Reduced granularity for public insights and community validation. Some location-based features may be less precise.

Decision 11: Separate Indoor and Outdoor Scoring Feature Sets in Recommendation

Decision: Use distinct feature sets for indoor and outdoor plant recommendations.

Rationale: Indoor and outdoor environments differ significantly in factors like light exposure, airflow, and humidity. Separating scoring models ensures that recommendations are contextually relevant and accurate. It allows tailored optimization for each environment rather than a one-size-fits-all approach.

Trade-off: Increased model complexity and maintenance overhead. Requires additional feature engineering and testing.

Decision 12: Partial Dashboard Response with Stale Indicators Instead of Full Fetch Failure

Decision: Return partial dashboard data with stale indicators when some services fail.

Rationale: Failing the entire dashboard due to a single service outage degrades user experience. Providing partial data ensures continuity and usability while signaling which components may be outdated. This approach improves resilience and perceived system reliability.

Trade-off: Users may see inconsistent or outdated information. Additional logic is needed to track and display staleness indicators.

4.2 User Interface Design

The interface was designed around three principles from the usability requirements: every primary piece of information on the dashboard is visible without scrolling on a desktop, the growth update flow takes five or fewer interactions from QR scan to confirmation, and verification states (pending, verified, rejected) are always shown with distinct labelled indicators. A green, nature-oriented palette is used, with orange reserved for pending/warning states and red for rejections and AQI alerts.

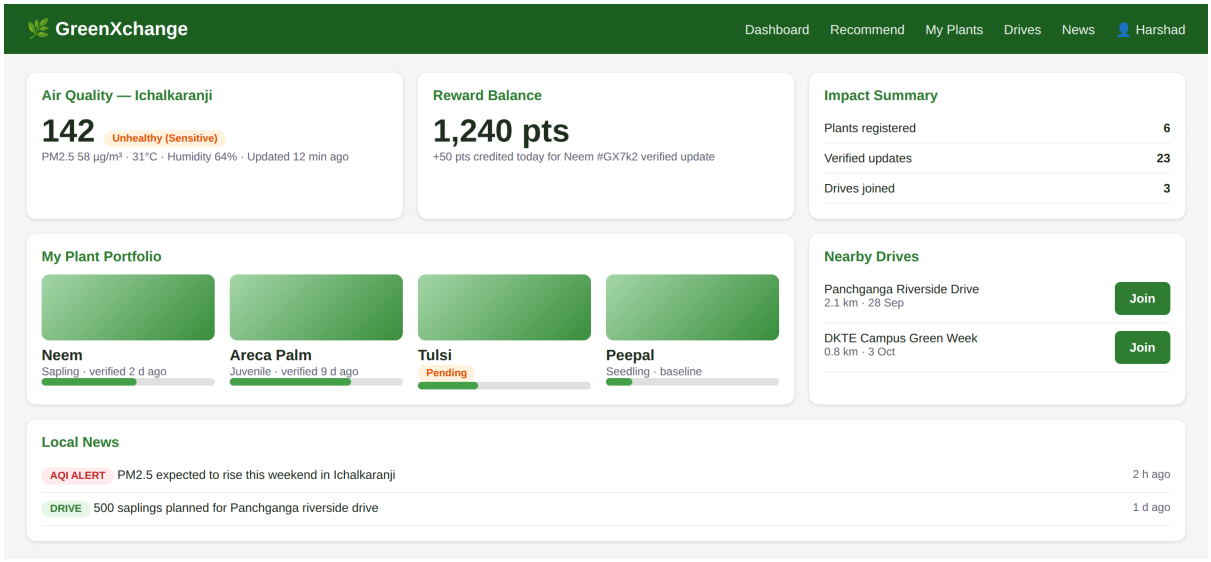


Figure 4.2: Dashboard: AQI panel, reward balance, plant portfolio, nearby drives and local news

The dashboard (Figure 4.2) aggregates the five panels defined in FR8. Each plant card shows its latest growth stage, last verified date and a progress bar. On mobile screens the panels stack vertically with the environmental panel first, and drives and news move into tabs.

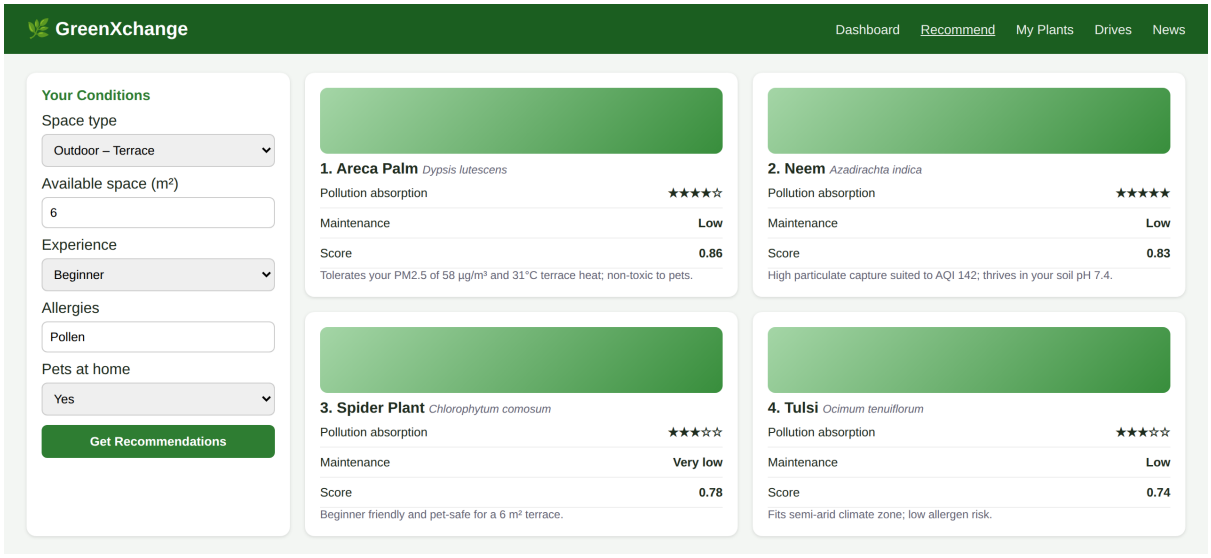


Figure 4.3: Recommendation screen with user constraints and explained species cards

The recommendation screen (Figure 4.3) collects the user's constraints on the left and returns ranked species cards on the right. Each card shows the common and scientific name, a visual pollution-absorption rating, maintenance level, composite score and a short explanation that quotes actual values from the user's environmental profile.

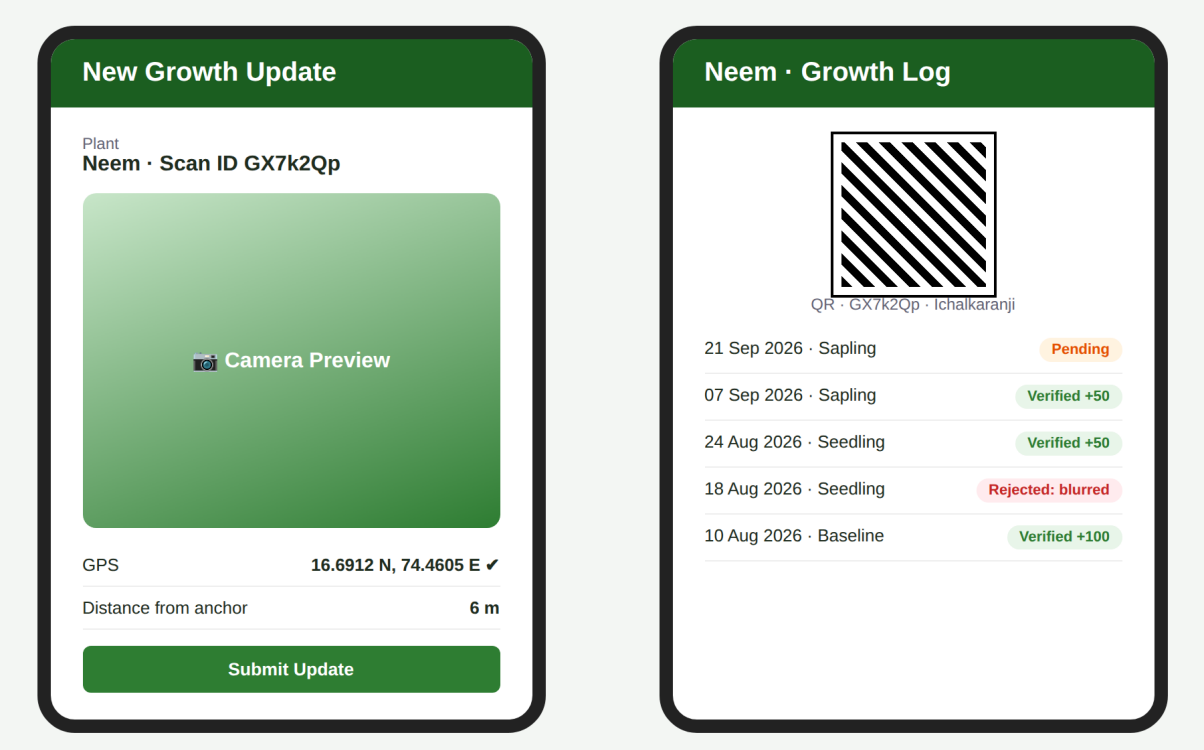


Figure 4.4: Mobile growth update capture (left) and plant growth log with QR identity (right)

The growth update screen (Figure 4.4, left) uses the in-app camera and shows the auto-filled GPS and the distance from the plant's anchor before submission. The growth log (right) shows the plant's QR identity with locality-level location only, and the status of every submission.

4.3 System Modeling

4.3.1 Data Flow Diagram

The Level 0 DFD (context diagram) treats GreenXchange as a single process exchanging data with the User, the Environmental Data Provider, the CV Verification Engine and the Admin. The Level 1 DFD decomposes it into nine processes: environmental data collection, recommendation, plant registration, growth update submission, verification processing (geo-anchor validation, CV image analysis and result consolidation), reward processing, community and drives, dashboard aggregation and the news feed service, together with the data stores D1 to D7.

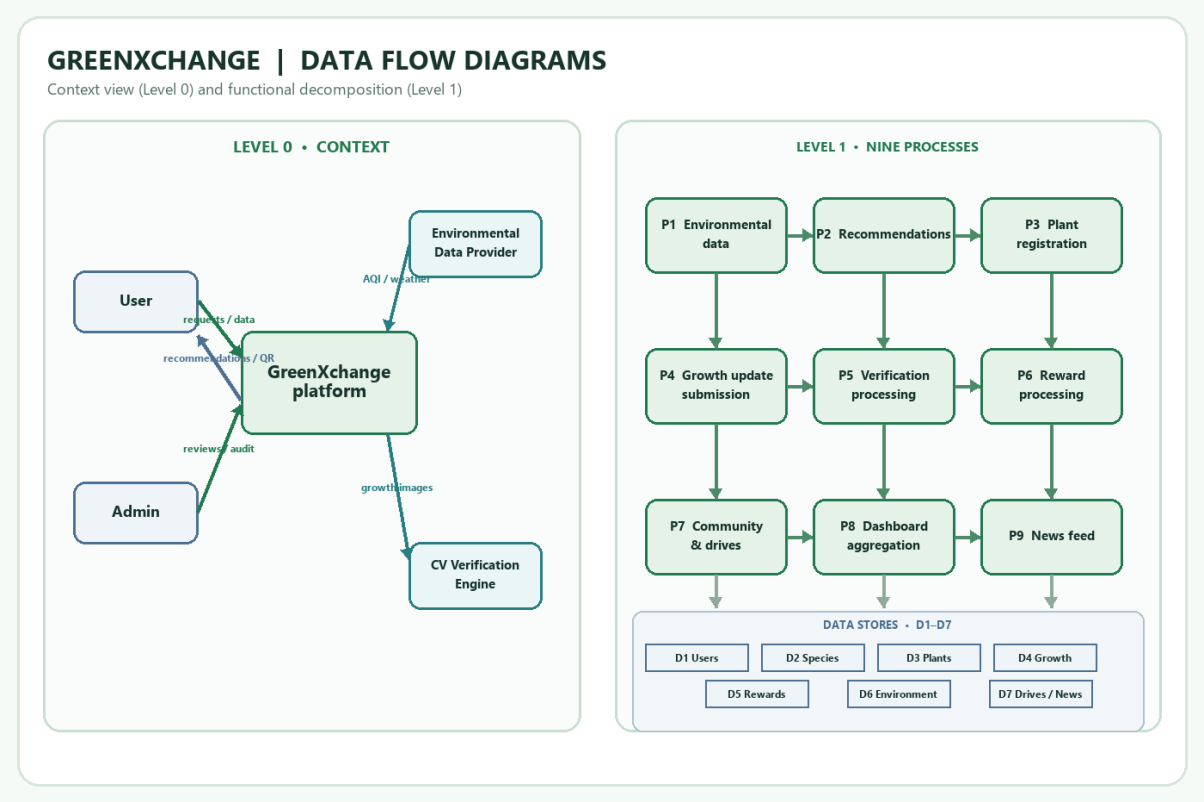


Figure 4.5: Level 0 and Level 1 data flow diagrams

4.3.2 Sequence Diagram

Figure 4.6 shows the sequence of interactions for the most important scenario, submitting a growth update. Geo-validation happens synchronously and cheaply before any image processing. The user receives a 202 Accepted response with a pending status while CV verification, reward crediting and notification happen asynchronously through the job queue and event bus.

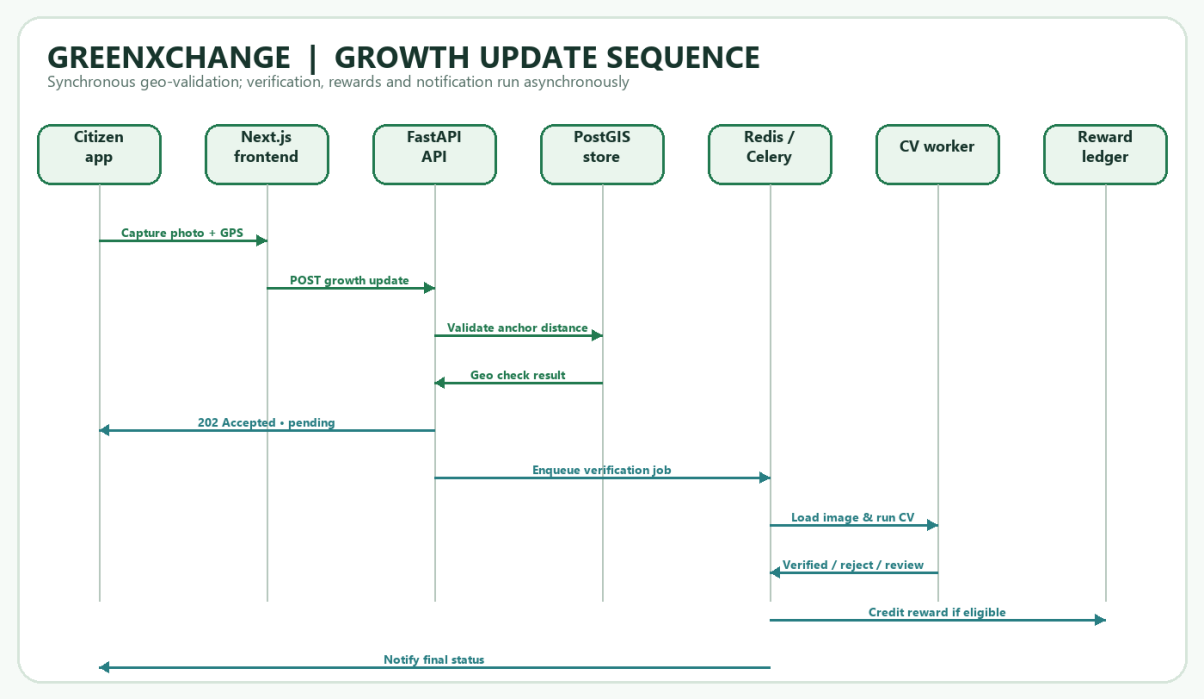


Figure 4.6: Sequence diagram for growth update submission and verification

4.3.3 Activity Diagram

The activity diagram in Figure 4.7 shows the decision flow of a growth update from capture to notification, including the three possible end states: rejected with a reason, flagged for manual review, or verified (with or without a reward, depending on the time gate).

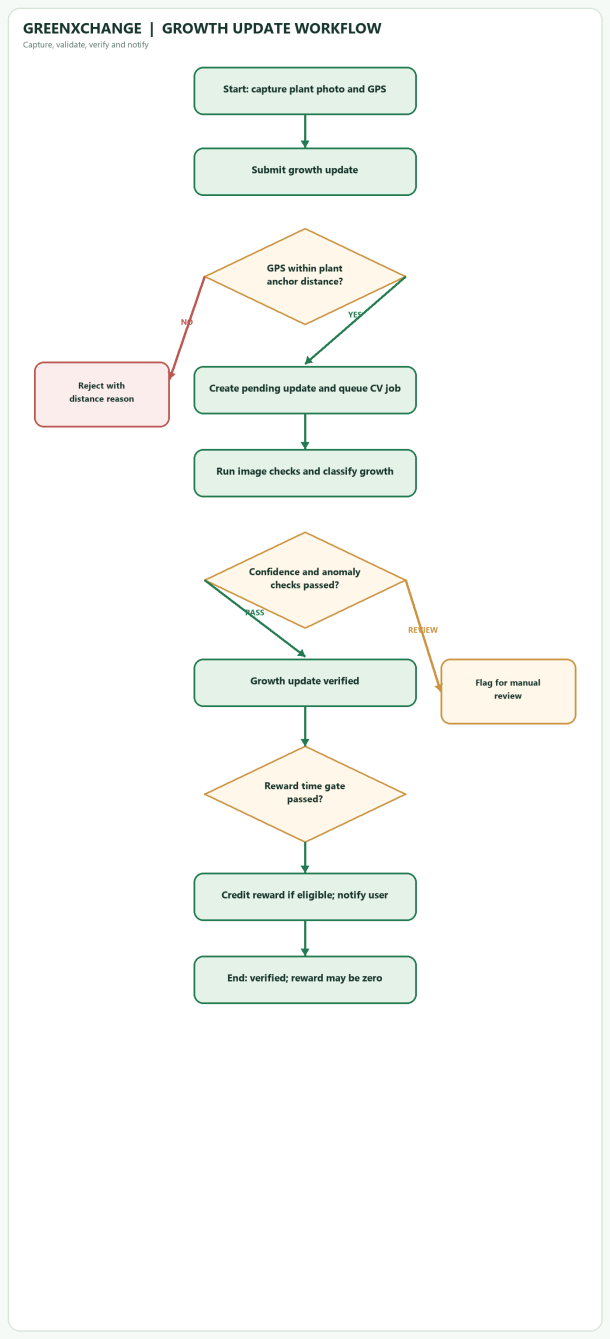


Figure 4.7: Activity diagram for the growth update workflow

4.3.4 Component Diagram

Figure 4.8 groups the software into five subsystems: the Next.js frontend, the Nginx and API gateway, the FastAPI application services, the Celery workers and the data layer. Services depend on the data layer only through repository interfaces, and on the workers only through queued jobs and published events.

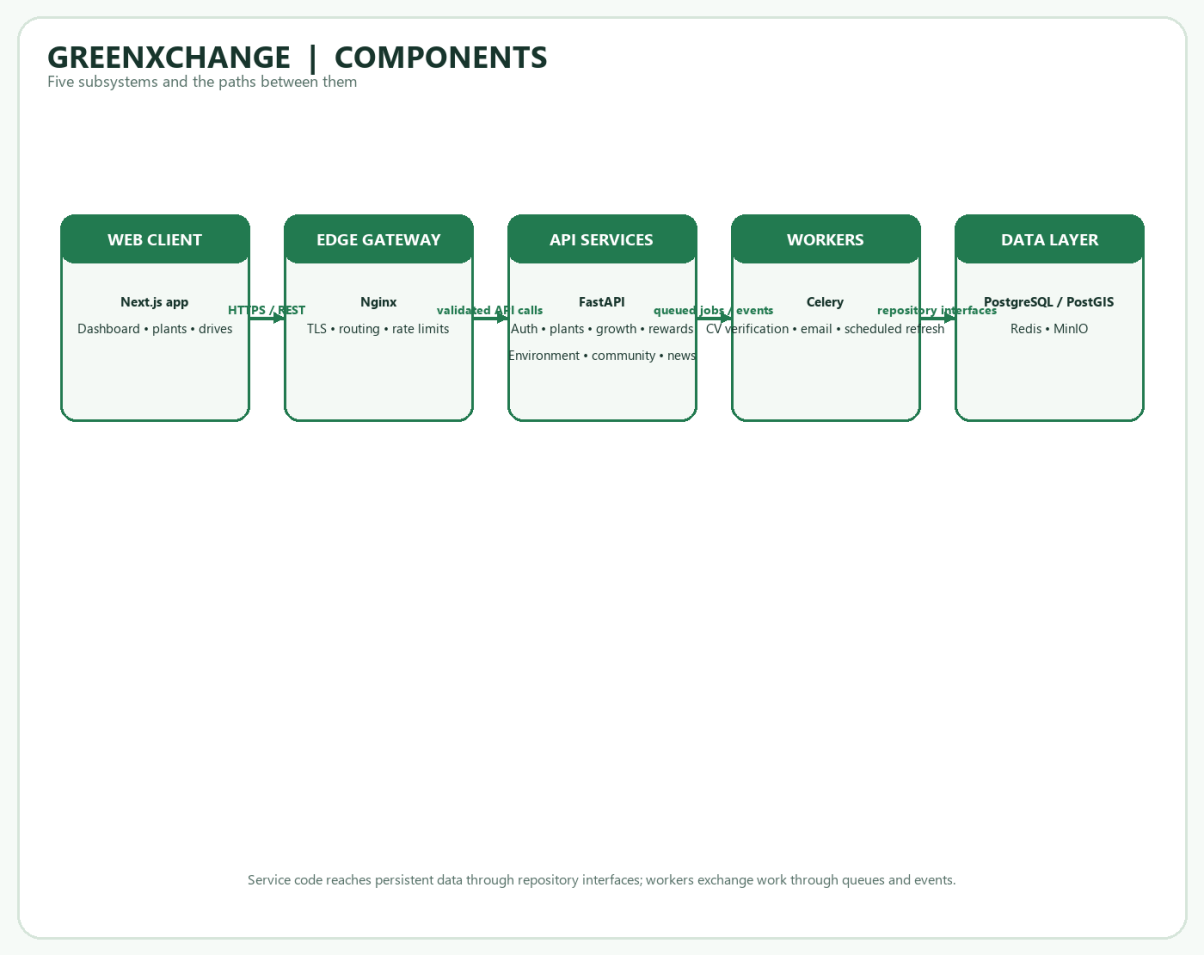


Figure 4.8: Component diagram

4.3.5 Deployment Diagram

The initial deployment (Figure 4.9) uses an application server hosting Nginx, the Next.js frontend, FastAPI instances and Celery workers as Docker containers, and a dedicated database server for PostgreSQL with PostGIS. Redis and object storage are reachable only on the private network, and only Nginx is exposed on port 443.

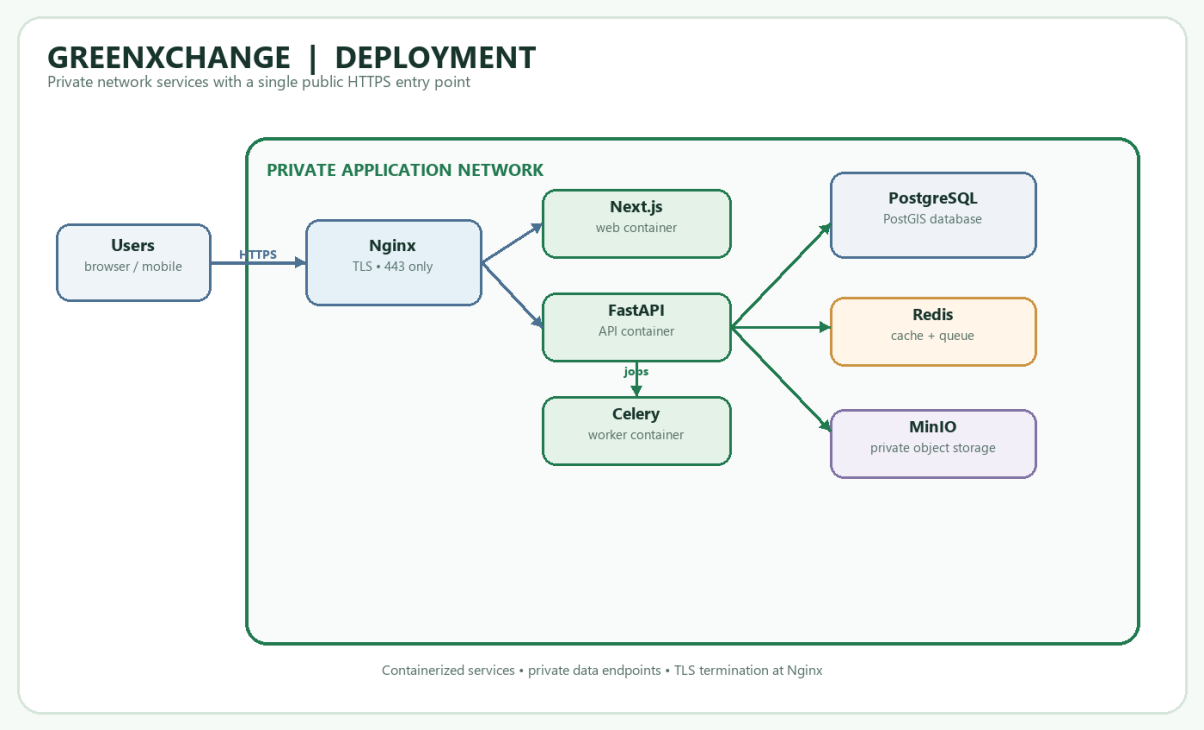


Figure 4.9: Deployment diagram

4.3.6 Class Diagram

The class diagram (Figure 4.10) models the core domain entities: User, Plant, GrowthUpdate, RewardTransaction, PlantSpecies, RecommendationRequest, EnvironmentalProfile, CommunityDrive, DriveParticipation and NewsFeedItem, with their attributes, operations and relationships.

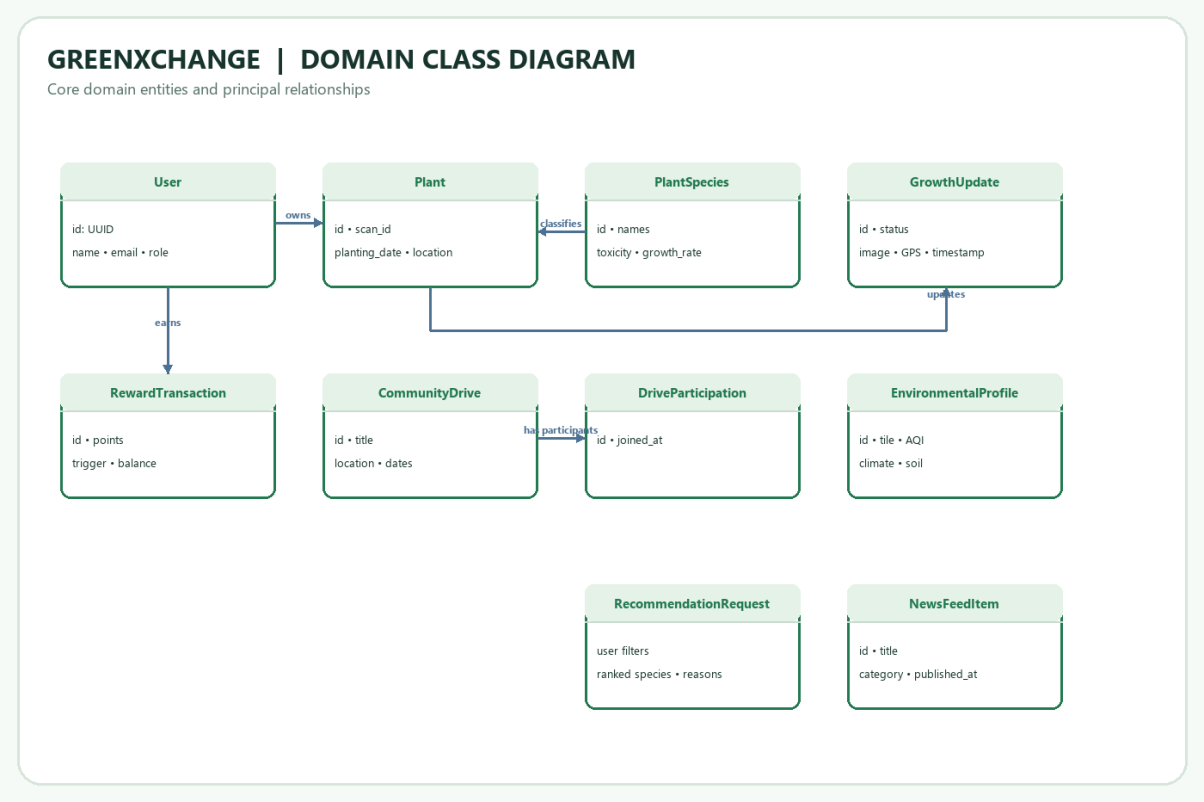


Figure 4.10: Class diagram

4.3.7 ER Diagram and Database Design

The entity-relationship diagram in crow's foot notation (Figure 4.11) shows nine tables and their keys. Spatial columns use the PostGIS GEOMETRY(POINT, 4326) and GEOMETRY(POLYGON, 4326) types with GiST indexes, and composite B-tree indexes support the most frequent queries such as the latest verified update per plant and the last reward per user and plant.

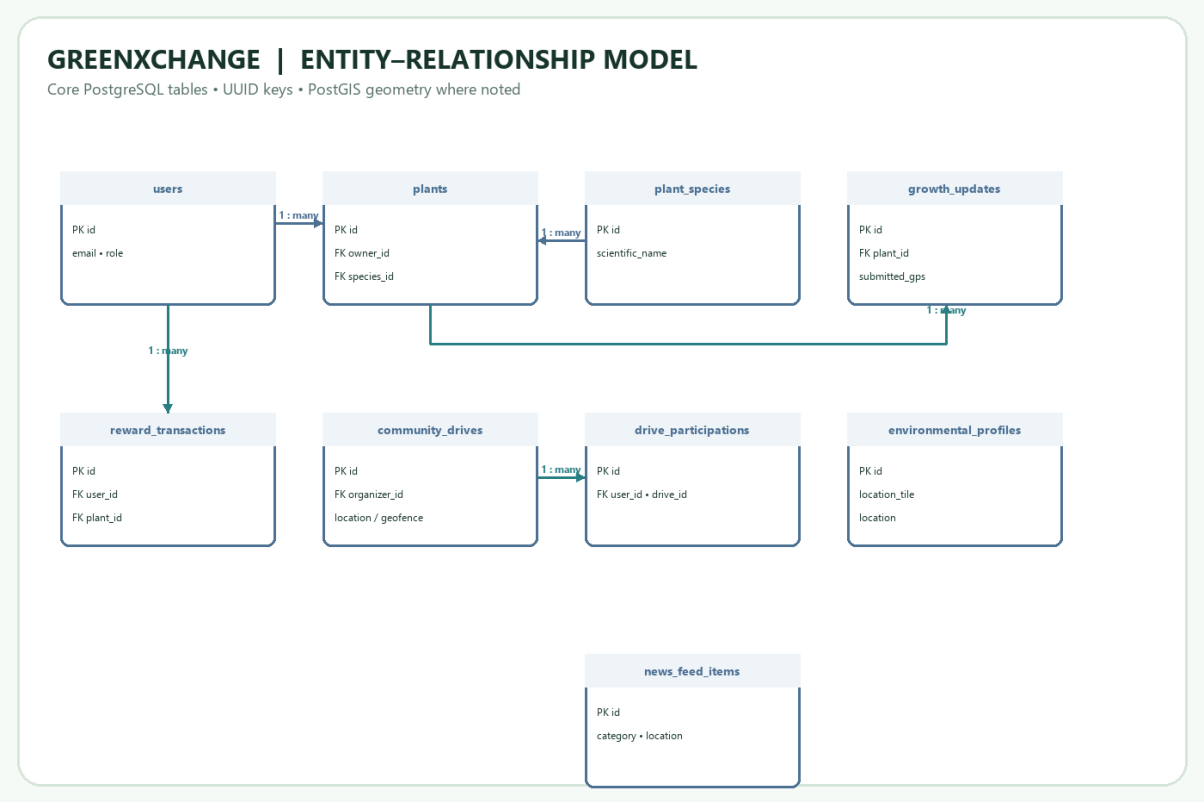


Figure 4.11: ER diagram (crow's foot notation)



Figure 4.12: Database schema: table definitions, indexing, enum types, retention and integrity rules

Table 4.1: Database tables

Table	Purpose	Key design notes
users	Accounts and profile	UUID PK, unique email, bcrypt/argon2 hash, device fingerprint
plant_species	Species knowledge base	Absorption rates, toxicity, allergen risk, pH and temperature ranges, space compatibility

Table 4.1: Database tables

Table	Purpose	Key design notes
plants	Registered physical plants	Unique scan_id, anchor_location POINT with GiST index, FK to users and species
growth_updates	Append-only growth log	Server timestamp, verification status enum, CV confidence, model version
reward_transactions	Append-only reward ledger	Balance snapshot per row, no UPDATE/DELETE, index on (user_id, plant_id, created_at)
community_drives	Plantation drives	location_center POINT and optional geofence POLYGON, participant counter
drive_participations	Drive membership	UNIQUE (user_id, drive_id) prevents duplicate joins
environmental_profiles	Cached location profile	Location tile key, derived pollution class and climate zone, TTL
news_feed_items	Local environmental news	Category enum, optional location scope, relevance score

4.3.8 Collaboration Diagram

Figure 4.13 shows the collaboration between objects for the two main flows, growth update verification and recommendation request, with numbered messages indicating the order of interaction.

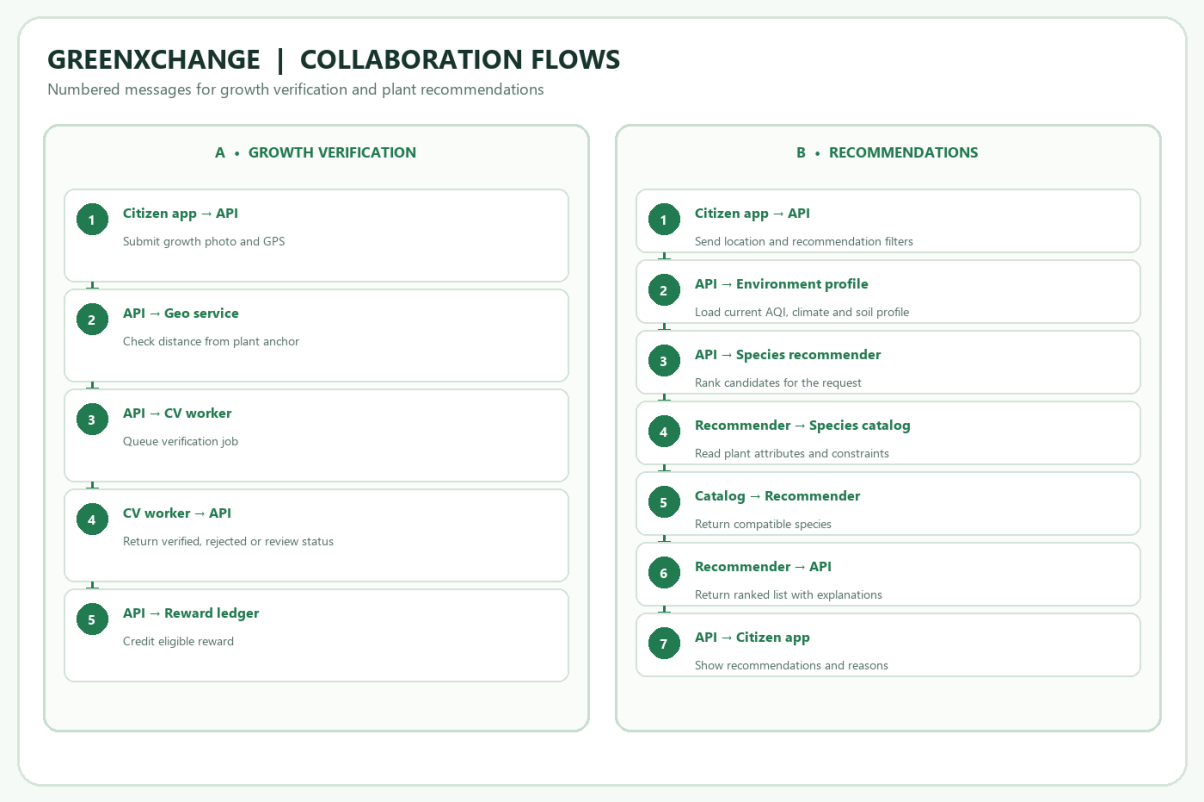


Figure 4.13: Collaboration diagrams for growth verification and recommendation flows

4.4 Algorithmic Description of Each Module

4.4.1 Environmental Profiling Module

For a request at (lat, lon) the module computes a tile key by rounding the coordinates to a fixed grid (about 1 km). If a non-expired profile exists in Redis under that key it is returned immediately. Otherwise the last stored profile is returned with a stale flag and a Celery task is queued to fetch fresh AQI and PM2.5 from WAQI and weather parameters from OpenWeatherMap, merge them with the soil attributes for the tile, compute the derived pollution-severity class (by binning AQI into the standard bands) and climate zone (from temperature and rainfall), and write the result back with a TTL per source (for example 30 minutes for AQI and 3 hours for weather). External APIs are therefore never called on the user's request path.

4.4.2 Algorithm 1: Recommendation Engine Pipeline

- **Inputs:** user_location (Point); user_filters {space_type, available_space_sqm, indoor_outdoor, experience_level, declared_allergies[], has_pets}
- **Outputs:** top N species with score and explanation, or an empty list with reason codes
- **Preconditions:** environmental profile cache available; plant species table accessible; AQI, climate and soil data valid

```
ALGORITHM RecommendationEnginePipeline(user_location, user_filters)
BEGIN
  environmental_profile ← FETCH_FROM_CACHE(user_location)
  pollution_class ← BIN_AQI(environmental_profile.AQI)
  climate_zone ← DERIVE_CLIMATE(environmental_profile.temperature,
                          environmental_profile.rainfall)
  species_list ← QUERY_ALL_PLANT_SPECIES()
  filtered_species ← EMPTY_LIST
FOR EACH species IN species_list DO
  IF NOT OVERLAP(species.min_temp, species.max_temp,
          environmental_profile.temperature_range)
    CONTINUE
  IF NOT OVERLAP(species.soil_ph_range,
          environmental_profile.soil_ph)
    CONTINUE
  IF NOT CONTAINS(species.space_type_compatibility,
           user_filters.space_type)
    CONTINUE
  IF user_filters.has_pets = TRUE AND
   species.toxicity_level IN {MEDIUM, HIGH}
    CONTINUE
  IF user_filters.declared_allergies IS NOT EMPTY AND
   (species.allergen_risk = HIGH OR
    SEASONAL_ALLERGEN(species) = HIGH)
    CONTINUE
  ADD species TO filtered_species
END FOR
IF filtered_species IS EMPTY THEN
       RETURN EMPTY_LIST WITH_REASON_CODES
  END IF
  scored_species ← EMPTY_LIST
  FOR EACH species IN filtered_species DO
       score ← 0.35 * species.pollution_absorption_score +
            0.25 * INVERSE(species.maintenance_score) +
            0.20 * GROWTH_MATCH(species, user_filters) +
            0.20 * ECOLOGICAL_FIT(species, climate_zone)
       explanation ← GENERATE_EXPLANATION(species, score)
       ADD (species, score, explanation) TO scored_species
  END FOR
  SORT scored_species BY score DESCENDING
  RETURN TOP_N(scored_species)
```

```
END
```

Edge cases and complexity: If every species is filtered out the algorithm returns an empty list with reason codes. There is no fallback to soft filtering. Scores are computed from normalized (0 to 1) attributes, so the weights 0.35, 0.25, 0.20 and 0.20 translate directly into relative importance. The complexity is O(n log n) for n species, which is negligible for a dataset of 500 to 2,000 species.

4.4.3 Algorithm 2: Growth Update Verification Pipeline

- **Inputs:** update_id, submitted_gps (Point), exif_gps (Point or NULL), image_binary, plant_id
- **Outputs:** verification_status (verified / rejected / manual-review) with confidence score, growth stage and reason
- **Preconditions:** plant record exists; CV worker queue operational; distance threshold configured

```
ALGORITHM GrowthUpdateVerification(update_id, submitted_gps, exif_gps,
    image_binary,
plant_id)
BEGIN
  plant_record ← FETCH_PLANT(plant_id)
  anchor_location ← plant_record.anchor_location
  distance ← DISTANCE(submitted_gps, anchor_location)
  IF distance > THRESHOLD THEN
       RETURN REJECTED WITH_REASON("LOCATION-MISMATCH")
  END IF
  IF exif_gps IS NOT NULL THEN
       delta ← DISTANCE(exif_gps, submitted_gps)
       IF delta > 200 METERS THEN
      FLAG_MANUAL_REVIEW(update_id)
      RETURN MANUAL_REVIEW
    END IF
  END IF
  ENQUEUE_TO_CV_QUEUE(update_id, image_binary)
  CV_RESULTS ← CV_WORKER_PROCESS(update_id)
  IF NO_PREVIOUS_VERIFIED_IMAGE(plant_id) THEN
    SKIP growth_stage_comparison
  END IF
  IF CV_RESULTS.ALL_PASS = TRUE AND GEO_CHECK_PASS = TRUE THEN
```

```
    UPDATE_STATUS(update_id, VERIFIED)
    STORE_METADATA(cv_confidence, growth_stage)
    EMIT_EVENT("verification-completed")
  ELSE
    UPDATE_STATUS(update_id, REJECTED)
    STORE_REJECTION_REASON(CV_RESULTS.failure_reason)
  END IF
  RETURN FINAL_STATUS
END
```

Edge cases: With no previously verified image, the submission is treated as a baseline and stage comparison is skipped. An EXIF/GPS mismatch sends the update to manual review instead of rejecting it.

4.4.4 Algorithm 3: Reward Eligibility and Credit Logic

- **Inputs:** user_id, plant_id, trigger_event
- **Outputs:** transaction_id and updated balance, or an ineligible status
- **Preconditions:** reward configuration exists; database supports row-level locking

```
ALGORITHM RewardEligibility(user_id, plant_id, trigger_event)
BEGIN
  START TRANSACTION WITH ROW LOCK
  last_txn ← FETCH_LAST_TRANSACTION(user_id, plant_id)
  elapsed_time ← CURRENT_TIME - last_txn.timestamp
  min_interval ← GET_MIN_INTERVAL(trigger_event)
  IF elapsed_time < min_interval THEN
       ROLLBACK
       RETURN INELIGIBLE("TIME-GATE")
  END IF
  IF NOT VALID_EVENT(trigger_event) THEN
       ROLLBACK
       RETURN INVALID_EVENT
  END IF
  points ← FETCH_POINTS(trigger_event)
  current_balance ← SUM_ALL_TRANSACTIONS(user_id)
  new_balance ← current_balance + points
  txn_id ← INSERT_TRANSACTION(user_id, plant_id,
                   trigger_event, points, new_balance)
  COMMIT
  RETURN txn_id, new_balance
```

```
END
```

Edge cases: Concurrent requests for the same user and plant are serialized by the row lock, so a double credit is impossible. Invalid events are rejected immediately.

4.4.5 Algorithm 4: Community Drive Discovery

```
ALGORITHM DiscoverDrives(user_location, radius_m, today)
BEGIN
  drives <- SELECT * FROM community_drives
            WHERE ST_DWithin(location_center::geography,
                             user_location::geography, radius_m)
              AND end_date >= today
            ORDER BY ST_Distance(location_center, user_location)
            LIMIT 20                    -- served by GiST index
  FOR EACH d IN drives DO
     d.joined <- EXISTS(drive_participations WHERE user_id = me AND drive_id = d.
    id)
  END FOR
  RETURN drives
END

ALGORITHM JoinDrive(user_id, drive_id)
BEGIN
  INSERT INTO drive_participations(user_id, drive_id)   -- UNIQUE(user_id,
    drive_id)
  ON CONFLICT DO NOTHING RETURNING id INTO pid
  IF pid IS NOT NULL THEN
     UPDATE community_drives SET participant_count = participant_count + 1
     WHERE drive_id = drive_id
  END IF
  RETURN pid IS NOT NULL
END
```

4.4.6 Algorithm 5: News Feed Relevance Scoring

```
ALGORITHM BuildFeed(user_location, category_filter)
BEGIN
  items <- recent news_feed_items (last 30 days) matching category_filter
  FOR EACH item IN items DO
```

```
    IF item.location_scope IS NULL THEN proximity <- 0.5      -- globally
   relevant
    ELSE proximity <- EXP(-DISTANCE_KM(item.scope_center, user_location) / 25)
    recency   <- EXP(-AGE_HOURS(item.published_at) / 72)
    cat_w     <- {local_aqi: 1.0, drive_announcement: 0.9, awareness: 0.6}[item.
   category]
    item.relevance <- 0.5*proximity + 0.3*recency + 0.2*cat_w
  END FOR
  SORT items BY relevance DESC, published_at DESC
  RETURN TOP 20 items
END
```

4.4.7 Algorithm 6: Dashboard Aggregation with Partial Availability

```
ALGORITHM Dashboard(user)
BEGIN
  tasks <- { env: GET_PROFILE(user.location), plants: GET_PORTFOLIO(user),
             rewards: GET_BALANCE(user), drives: DISCOVER_DRIVES(user.location),
             news: BUILD_FEED(user.location) }
  results <- RUN_IN_PARALLEL(tasks, timeout = 2 s per task)
  FOR EACH (panel, r) IN results DO
     IF r TIMED_OUT OR FAILED THEN
        response[panel] <- LAST_CACHED(panel, user); response[panel].stale <- TRUE
     ELSE response[panel] <- r; response[panel].stale <- FALSE
  END FOR
  RETURN response                    -- always returned within about 3 s
END
```

CHAPTER 5: IMPLEMENTATION

5.1 Environmental Setting for Running the Project

5.1.1 Technology Stack

Programming Languages

Python 3.11 — Backend Services and ML Pipeline Python is the primary language for all backend services, the recommendation engine, the data ingestion and caching layer, and the computer vision processing pipeline. Python's ecosystem provides mature, production-grade libraries for every component of the backend including web framework, async task processing, machine learning, computer vision, and geospatial computation. Version 3.11 is selected for its performance improvements over prior versions and its stability for production deployment.

JavaScript and TypeScript — Frontend Web Application TypeScript is used for the frontend web application to enforce type safety across component interfaces, API response handling, and state management logic. Type enforcement at the frontend layer reduces runtime errors in a dashboard- heavy application where API response shapes are complex and aggregated from multiple backend modules.

SQL — Database Query Layer PostgreSQL-dialect SQL is used for all relational database interactions, spatial queries via PostGIS, and schema definition. Stored procedures are avoided in favor of service-layer business logic to maintain testability and separation of concerns.

Frameworks

FastAPI — Backend API Framework FastAPI provides asynchronous request handling, automatic OpenAPI schema generation, native Pydantic integration for request and response validation, and dependency injection. For GreenXchange, FastAPI's async support is critical because the API layer must handle growth update submissions that immediately enqueue to an async job queue without blocking the response, and dashboard aggregation that executes multiple data fetches in parallel.

React with Next.js — Frontend Framework Next.js provides server-side rendering for initial dashboard load performance, file-based routing for the multi-module frontend structure, and API route capabilities for backend-for-frontend patterns where dashboard aggregation calls can be proxied. React's component model supports the modular dashboard structure with independently refreshable panels for environmental data, plant portfolio, community drives, and news feed.

Celery — Async Task Queue Worker Celery manages the asynchronous execution of CV verification jobs, handling task queuing, worker scaling, retry logic on failure, and task state tracking. Celery is used in conjunction with Redis as the message broker and result backend, providing a mature, production-tested async processing infrastructure for the verification pipeline.

SQLAlchemy — ORM and Database Abstraction SQLAlchemy is used as the ORM layer for all relational database interactions, providing query construction, connection pooling, and transaction management. SQLAlchemy Core is used for performance-critical queries where ORM overhead is unacceptable, while the ORM layer handles standard entity operations.

Database

PostgreSQL 15 with PostGIS Extension — Primary Relational Database PostgreSQL is the primary data store for all relational entities including users, plants, growth updates, reward transactions, community drives, and species data. The PostGIS extension provides native spatial data type support for GEOMETRY columns, GiST spatial indexing, and spatial query functions including ST_DWithin used for drive discovery and geo-validation proximity calculations. PostgreSQL 15 is selected for its improvements to logical replication, JSONB performance, and range type handling, all of which are relevant to GreenXchange's data model.

Redis — Cache Layer and Message Broker Redis serves dual roles. As a cache layer it stores environmental profile data with TTL expiry, eliminating per-request external API calls for recommendation generation. As a message broker it backs the Celery task queue for CV verification job distribution to workers. Redis is selected for its sub-millisecond read performance, native TTL support on cache entries, and proven reliability as a Celery broker.

Amazon S3 or Compatible Object Storage — Image Storage All growth update images are stored in object storage after server-side re-encoding and EXIF metadata stripping. Object storage is selected over filesystem or database BLOB storage for scalability, direct URL delivery, and separation of binary asset storage from relational data. An S3-compatible API allows the deployment to use AWS S3 in production or MinIO in self-hosted environments without code changes.

Tools and Platforms

Docker and Docker Compose — Containerization and Local Orchestration All services including the FastAPI application, Celery workers, PostgreSQL database, Redis instance, and object storage are containerized using Docker. Docker Compose orchestrates the full local development environment, ensuring environment parity between development and production and eliminating dependency conflicts across services.

Celery with Redis Broker — Async Job Processing Infrastructure As described under frameworks, Celery with Redis broker manages the async CV verification pipeline. Worker

instances can be scaled horizontally by adding additional Celery worker containers without changes to the application code.

OpenCV and PyTorch — Computer Vision Processing OpenCV is used for image preprocessing, error level analysis for manipulation detection, and EXIF metadata extraction at ingestion. PyTorch backs the trained models for plant object detection and growth stage classification. A YOLO-architecture model fine-tuned on plant imagery handles plant presence detection, and a classification model handles growth stage labeling against prior verified images.

PostGIS — Spatial Database Extension PostGIS extends PostgreSQL with spatial data types, GiST indexing on geometry columns, and spatial functions. ST_DWithin is used for community drive proximity queries. Haversine-based distance computation is used for geovalidation.

OpenWeatherMap and WAQI APIs — Environmental Data Providers OpenWeatherMap provides weather parameter data including temperature, humidity, rainfall, and sunlight estimation. The World Air Quality Index API provides AQI and PM2.5 data. Both are accessed through the environmental data service layer and results are cached in Redis with source-appropriate TTL values rather than queried per user request.

JWT — Authentication JSON Web Tokens are used for stateless authentication across all API endpoints. Tokens are issued at login and validated at the API gateway level before routing to application services. Token expiry and refresh logic follow standard security practices with short- lived access tokens and longer-lived refresh tokens.

Nginx — Reverse Proxy and Static Asset Delivery Nginx serves as the reverse proxy in front of the FastAPI application, handling SSL termination, request routing, static asset delivery for the frontend, and connection load distribution across application server instances.

Justification

The technology stack is selected on four criteria applied consistently across every component: functional fitness for the specific role the component plays in the GreenXchange architecture, maturity and production reliability of the library or framework, compatibility with the self-hosted and open-source tooling preference that minimizes external service dependency and operational cost, and alignment with the Python and JavaScript skill domains appropriate to the development context. PostgreSQL with PostGIS is non-negotiable for this system. No other open-source relational database provides the combination of mature spatial indexing, spatial query functions, JSONB support for semi-structured species attributes, and append-only table semantics required by the growth log and reward ledger. A NoSQL alternative would require sacrificing transactional integrity on the reward ledger, which is architecturally unacceptable. FastAPI over Django REST Framework is selected because GreenXchange's API layer is not CRUD-heavy in the traditional sense. The most performance-sensitive endpoints involve async job enqueueing on growth update submission and parallel data aggregation on dashboard fetch.

FastAPI's native async support handles both patterns without the synchronous request handling constraints of Django's ORM layer.

Celery over a cloud-managed queue service is selected to maintain self-hosted deployment capability and avoid vendor lock-in on the CV verification pipeline, which is the system's most computationally intensive component and the one most likely to require custom scaling behavior. Redis serving both cache and broker roles is a deliberate consolidation decision. Operating a single Redis instance for both functions reduces infrastructure complexity at the cost of requiring careful key namespace management, which is a tractable operational concern relative to the complexity of managing separate caching and messaging infrastructure. React with Next.js over a single-page application framework is selected because the dashboard's initial load performance is critical to user retention. Server-side rendering of the initial dashboard state eliminates the blank loading screen that client-side-only applications produce, which is particularly important for a dashboard that aggregates data from multiple backend modules on every load.

5.1.2 Hardware Requirements

Processor: Intel Core i5 eighth generation or above, or AMD Ryzen 5 3000 series or above, with a minimum of four physical cores. Multi-core capacity is required because the local development environment runs the FastAPI server, Celery workers, PostgreSQL, Redis, and the frontend development server simultaneously as Docker containers.

RAM: Minimum sixteen gigabytes. PyTorch model loading for CV inference during local testing consumes approximately two to four gigabytes depending on model size. PostgreSQL with PostGIS requires a minimum of two gigabytes under development load. The remaining capacity supports the application server, Redis, and operating system overhead.

Storage: Minimum 256 gigabytes SSD. Docker images for all services accumulate to approximately eight to twelve gigabytes. Plant species datasets, training image datasets for CV model development, and growth update images stored locally during development require substantial storage headroom. SSD storage is required for acceptable PostgreSQL and Redis performance.

GPU: Optional for development but strongly recommended for CV model training and local inference. An NVIDIA GPU with CUDA support and a minimum of four gigabytes of VRAM significantly reduces model training time and inference latency during development testing. PyTorch supports CPU-only inference as a fallback.

Production Environment — Minimum Specification for Initial Deployment

Application Server: Virtual machine or physical server with four virtual CPUs and eight gigabytes of RAM. This instance hosts the FastAPI application server behind Nginx and two to four Celery worker processes for CV verification.

Database Server: Virtual machine or physical server with two virtual CPUs and eight gi-

gabytes of RAM dedicated to PostgreSQL with PostGIS. Dedicated database hosting is required to prevent resource contention between the database and application workloads.

Storage: minimum one hundred gigabytes SSD for database files with growth headroom for the append-only growth log and reward ledger.

Object Storage: S3-compatible storage with no fixed minimum capacity, scaling with image upload volume. Estimated at five to ten kilobytes per re-encoded image after compression, accumulating based on user submission rate.

Redis Instance: One gigabyte of RAM minimum, accommodating the environmental profile cache, Celery task queue, and task result backend simultaneously.

5.1.3 Software Requirements

Operating System

Production: Ubuntu Server 22.04 LTS. Ubuntu 22.04 is selected for its long-term support status, broad compatibility with all required software packages including PostGIS, Docker Engine, and NVIDIA CUDA drivers for GPU-enabled workers, and its active security update cadence.

Development: Ubuntu 22.04, macOS Ventura or above, or Windows 11 with WSL2 running Ubuntu 22.04. Docker Desktop is required on macOS and Windows to provide the Linux container runtime environment.

Runtime Environments

Python 3.11 is required for the backend application and CV worker. PyPy is not compatible with PyTorch and must not be used. Node.js 18 LTS or above is required for the Next.js frontend build and development server.

Core Software Dependencies

FastAPI 0.100 or above. Uvicorn as the ASGI server for FastAPI. SQLAlchemy 2.0 or above with asyncpg driver for async PostgreSQL connectivity. Alembic for database migration management. Celery 5.3 or above with Redis as broker and result backend. OpenCV 4.8 or above for image preprocessing and ELA. PyTorch 2.0 or above for CV model inference.

Ultralytics YOLOv8 or above for plant detection model. React 18 or above. Next.js 14 or above. Docker Engine 24 or above. Docker Compose 2.20 or above. Nginx 1.24 or above.

Database Software

PostgreSQL 15 or above is required. The PostGIS extension version 3.3 or above must be installed on the PostgreSQL instance. The pgcrypto extension must be enabled for UUID generation using gen_random_uuid(). Redis 7.0 or above is required for both the cache layer and the Celery broker. Redis must be configured with maxmemory-policy set to allkeys-lru to manage memory under cache load.

Development and Build Tools

Git for version control. pip and virtualenv or poetry for Python dependency management.

npm or yarn for frontend dependency management. Alembic CLI for database migration execution. Docker CLI for container management. Pytest for backend unit and integration testing. Jest and React Testing Library for frontend component testing.

5.1.4 Network Requirements

Bandwidth: The production server must have a minimum upstream bandwidth of one hundred megabits per second to support concurrent image uploads from multiple users without upload queue saturation. Growth update image uploads after server-side compression are estimated at an average of two to five megabytes per submission. At fifty concurrent uploads, peak upstream consumption is approximately twenty to forty megabits per second, requiring the one hundred megabit upstream minimum. Downstream bandwidth must support dashboard data delivery and recommendation responses to concurrent users. Dashboard responses are JSON payloads of approximately ten to fifty kilobytes per user, making downstream bandwidth a lower constraint than upstream.

Latency: Internal network latency between the application server and the PostgreSQL database must not exceed five milliseconds. Database query latency is a primary component of API response time, and high internal network latency directly impacts response time SLAs. Co-located deployment on the same physical server or local network is strongly preferred over geographically separated instances. Internal latency between the application server and the Redis instance must not exceed two milliseconds, as cache reads are on the critical path of every recommendation request. External API calls to OpenWeatherMap and WAQI are not on the critical path of any user-facing request because all environmental data is served from cache. External API latency affects only the background cache refresh task and is therefore not a user-facing latency concern.

Connectivity: The production server must have reliable public internet connectivity for external API polling of environmental data providers and for delivery of push notifications to users. Intermittent external connectivity must not cause user-facing failures because environmental data is served from cache with defined TTL and stale indicator fallback. The PostgreSQL database instance must not be directly reachable from the public internet. All database connections must route through the application server layer. A firewall rule or security group must restrict PostgreSQL port access to the application server's internal IP address exclusively. The Redis instance must similarly be restricted to internal network access only.

5.2 Detailed Description of Methods

5.2.1 Haversine Distance for Geo-Anchor Validation

The distance d between the submitted point (φ1, λ1) and the registered anchor (φ2, λ2), in radians, on a sphere of radius R = 6,371 km is computed as:

$$a = \sin^2\left(\frac{\varphi_2 - \varphi_1}{2}\right) + \cos\varphi_1 \, \cos\varphi_2 \, \sin^2\left(\frac{\lambda_2 - \lambda_1}{2}\right)$$
$$d = 2R \cdot \text{atan2}\left(\sqrt{a}, \sqrt{1-a}\right)$$

A submission is accepted for CV processing only if d is within the configured tolerance radius (default 25 m, which allows for normal smartphone GPS error). The same function computes the EXIF-to-submitted GPS delta, where a value above 200 m sends the update to manual review.

5.2.2 Weighted Multi-Criteria Scoring

All species attributes are min-max normalized to the range 0 to 1 over the species table. For each species surviving the hard filter the composite score is:

$$\textit{score} = 0.35 \cdot \textit{absorption}_{norm} + 0.25 \cdot (1 - \textit{maintenance}_{norm}) + 0.20 \cdot \textit{growth_match} + 0.20 \cdot \textit{ecological_fit}$$

For outdoor spaces absorption_norm is the mean of the normalized PM2.5, SO_2 and NOx absorption rates. For indoor spaces the indoor VOC absorption rate is used instead, which gives separate indoor and outdoor feature sets (Design Decision 11). growth_match rewards fast-growing species for large spaces and slow-growing ones for small spaces and beginners. ecological_fit is 1.0 when the species is native to or recorded in the user's climate zone and decreases with distance from its preferred temperature and pH centre. The explanation string is generated from a template that inserts the user's actual AQI, PM2.5, temperature and pH values alongside the attribute that contributed most to the score.

5.2.3 Error Level Analysis (ELA)

The uploaded JPEG is re-saved at 90% quality with OpenCV and the absolute per-pixel difference from the original is computed and amplified. For an unedited photo the error level is roughly uniform, while pasted or retouched regions show a markedly different level. The mean and 99th-percentile error, and the ratio between the highest-error block and the image median,

are compared with thresholds calibrated on a set of genuine and deliberately edited images. Images that are screenshots, heavily recompressed downloads or locally edited composites are rejected as MANIPULATION-SUSPECTED.

5.2.4 Plant Presence Detection (YOLOv8)

A YOLOv8-small model initialized from COCO-pretrained weights was fine-tuned on plant images collected by the team and from open datasets, with bounding boxes for potted plants, saplings and trees. Images are resized to 640 x 640. A submission passes if at least one plant box is detected with confidence of 0.5 or higher and covers a minimum fraction of the frame, which rejects photos where the plant is absent or too distant to assess.

5.2.5 Growth Stage Classification and Species Continuity

The largest detected plant region is cropped and passed to a ResNet-50 classifier fine-tuned with transfer learning to five stages: seedling, sapling, juvenile, mature and flowering/fruiting. The predicted stage must be the same as, or later than, the stage of the most recent verified image for that plant (a decrease is allowed only if the confidence is low, in which case the update goes to manual review). For species continuity, embeddings from the classifier's penultimate layer are compared by cosine similarity with the plant's previous verified images. A similarity below the threshold indicates that a different plant has been photographed.

5.2.6 Scan ID Generation

After the plant row is committed, the server takes the first 64 bits of the plant's UUID and encodes them in base62 (0-9, A-Z, a-z), giving an 11-character, URL-safe identifier such as GX7k2QpLm0e. A UNIQUE constraint on scan_id guards against the negligible chance of collision, and the QR code encodes the public URL https://<host>/p/<scan_id>. The ID is generated server-side only, after the database write, so a client cannot choose or pre-generate identifiers.

5.2.7 Time-Gate and Anti-Fraud Controls

The minimum interval between rewards for the same user and plant is configured per event type (for example 7 days for a growth update and a one-time bonus for the baseline registration). The check and the insert run in one transaction with SELECT ... FOR UPDATE on the user's latest ledger row. Additional controls include device fingerprinting at login and submission, one scan ID bound to one owner account, rate limiting of registration and submission endpoints, server-assigned timestamps, and re-encoding of every uploaded image (which also strips all EXIF data other than the GPS fields that are read before stripping).

5.3 Implementation Details

5.3.1 Project Structure

```
greenxchange/├──
 backend/│
   ├── app/│
   │   ├── main.py                # FastAPI app, routers, middleware│
   │   ├── core/                  # config, security (JWT), logging│
   │   ├── db/                    # SQLAlchemy models, session, repositories│
   │   ├── routers/               # env, recommend, plants, updates, rewards,│
   │   │                          # drives, news, dashboard, admin│
   │   ├── services/              # business logic per module│
   │   ├── workers/               # celery_app.py, cv_tasks.py, cache_tasks.py│
   │   ├── cv/                    # ela.py, detector.py, stage_classifier.py│
   │   └── events/                # Redis pub/sub event bus│
   ├── alembic/                   # versioned migrations and species seed data│
   ├── models/                    # yolov8s_plants.pt, resnet50_stage.pt│
   └── tests/                     # pytest unit and integration tests├──
 frontend/                       # Next.js 14 + TypeScript│
   ├── app/ (dashboard, recommend, plants, drives, news, p/[scanId])│
   └── components/├──
 nginx/nginx.conf├──
 docker-compose.yml└──
 .env.example
```

5.3.2 REST API Endpoints

Table 5.1: Main REST API endpoints

Method	Endpoint	Description	Auth
POST	/api/auth/register, /api/auth/login, /api/auth/refresh	Account creation, login and token refresh	Public
GET	/api/env/profile?lat=&lon=	Environmental profile for a location	JWT
POST	/api/recommendations	Ranked species list for profile and filters	JWT
POST	/api/plants	Register plant, returns plant_id, scan_id and QR	JWT
GET	/api/plants/{id}/growth-log	Owner view of growth history	JWT

Table 5.1: Main REST API endpoints

Method	Endpoint	Description	Auth
GET	/api/public/plants/{scan_id}	Public read-only profile (locality only)	Public
POST	/api/plants/{id}/updates	Submit growth update (multipart image + GPS)	JWT
GET	/api/updates/{update_id}	Verification status of an update	JWT
GET	/api/rewards/balance, /api/rewards/-transactions	Reward balance and ledger history	JWT
POST / GET	/api/drives, /api/drives/nearby	Create drive (organizer) / discover nearby drives	JWT
POST	/api/drives/{id}/join	Join a drive	JWT
GET	/api/news/feed	Personalized, location-scoped news	JWT
GET	/api/dashboard	Aggregated five-panel dashboard	JWT
GET / POST	/api/admin/reviews, /api/admin/audit	Manual review queue and audit logs	JWT + admin

5.3.3 Key Code Segments

Geo-validation (Haversine)

```python
from math import radians, sin, cos, atan2, sqrt
EARTH_RADIUS_M = 6_371_000

def haversine_m(lat1, lon1, lat2, lon2) -> float:
    p1, p2 = radians(lat1), radians(lat2)
    dphi, dlmb = p2 - p1, radians(lon2 - lon1)
    a = sin(dphi / 2) ** 2 + cos(p1) * cos(p2) * sin(dlmb / 2) ** 2
    return 2 * EARTH_RADIUS_M * atan2(sqrt(a), sqrt(1 - a))

def validate_anchor(plant, lat, lon, tol_m=settings.GEO_TOLERANCE_M):
    d = haversine_m(lat, lon, plant.anchor_lat, plant.anchor_lon)
    if d > tol_m:
        raise LocationMismatch(distance_m=round(d, 1))
    return d
```

Growth update endpoint (asynchronous hand-off)

```python
@router.post("/plants/{plant_id}/updates", status_code=202)
```

```
async def submit_update(plant_id: UUID, lat: float = Form(...), lon: float = Form
    (...),
                        image: UploadFile = File(...), user=Depends(current_user),
                        db: AsyncSession = Depends(get_db)):
    plant = await plants.get_owned(db, plant_id, user.id)
    validate_anchor(plant, lat, lon)                        # fail fast, no CV cost
    raw = await image.read()
    exif_gps = extract_exif_gps(raw)                        # read before stripping
    key = await storage.put(reencode_strip_exif(raw))       # original is discarded
    upd = await updates.create_pending(db, plant, user, lat, lon, exif_gps, key)
    verify_growth_update.delay(str(upd.id))                 # Celery task
    return {"update_id": upd.id, "status": "pending"}
```

Reward credit with row-level locking

```
async def credit_reward(db, user_id, plant_id, event):
    async with db.begin():
        last = (await db.execute(
            select(RewardTxn).where(RewardTxn.user_id == user_id)
            .order_by(RewardTxn.created_at.desc()).limit(1)
            .with_for_update())).scalar_one_or_none()
        last_for_plant = await ledger.last_for_plant(db, user_id, plant_id)
        if last_for_plant and now() - last_for_plant.created_at < MIN_INTERVAL[
    event]:
            log.info("reward_ineligible", reason="TIME-GATE", user=user_id, plant=
    plant_id)
            return None
        pts = POINTS[event]
        balance = (last.balance_snapshot if last else 0) + pts
        txn = RewardTxn(user_id=user_id, plant_id=plant_id, trigger_event=event,
                        points=pts, balance_snapshot=balance)
        db.add(txn)
    await bus.publish("reward.credited", txn.to_event())   # only after commit
    return txn
```

Dashboard aggregation

```
async def with_timeout(name, coro, user):
    try:
        return name, {"data": await asyncio.wait_for(coro, 2.0), "stale": False}
```

```
    except Exception:
        return name, {"data": await cache.last_panel(name, user.id), "stale": True}


@router.get("/dashboard")
async def dashboard(user=Depends(current_user)):
    jobs = {"env": env.profile(user.location), "plants": plants.portfolio(user.id),

            "rewards": rewards.balance(user.id), "drives": drives.nearby(user.
    location),
            "news": news.feed(user)}
    return dict(await asyncio.gather(*(with_timeout(k, v, user) for k, v in jobs.
    items())))
```

CHAPTER 6: INTEGRATION AND TESTING

6.1 Description of the Integration Modules

The modules were developed and unit-tested independently and then integrated incrementally in the order of their dependencies (bottom-up), so that each new module was integrated against modules that had already been verified.

Table 6.1: Module integration sequence

Step	Modules integrated	Integration mechanism	What was verified
1	Environmental Data Service + Redis cache + external APIs	Celery beat refresh task, Redis keys per location tile with TTL	Cache hit/miss path, stale flag, TTL expiry, API failure fallback
2	Recommendation Engine + Environmental Profile + species table	Service call and SQLAlchemy repository	Hard filters applied before scoring; empty list with reason codes
3	Plant Registration + PostGIS + QR generator	Transaction: insert plant, then generate scan_id and QR	scan_id only after commit; uniqueness; locality-only public view
4	Growth Update API + GeoValidation + object storage + job queue	Synchronous geo check, then image re-encode, S3 put, Celery enqueue	LOCATION-MISMATCH before CV; 202 pending response; original image discarded
5	CV Worker + growth log + event bus	Celery task writes result, publishes verification.completed	ELA, detection, stage and continuity checks; retries and dead-letter queue
6	Reward Ledger + event bus + notifications	Redis pub/sub subscriber	Credit only on verified events; time gate; row locking; publish after commit
7	Community Drives + News Feed	PostGIS ST_DWithin queries, relevance scoring	Proximity ordering, duplicate join prevention, feed ranking
8	Dashboard Aggregator + all modules + Next.js frontend	asyncio.gather with per-panel timeouts, REST to frontend	Five panels, partial response with stale indicators

6.2 Testing

6.2.1 Testing Strategy

- **Unit testing:** Pytest for backend services (scoring, hard filters, Haversine, base62 scan ID, time gate, relevance scoring) and Jest with React Testing Library for frontend components.

- **Integration testing:** Pytest with a Docker Compose test stack (PostgreSQL + PostGIS, Redis, MinIO) exercising API-to-database, API-to-queue and worker-to-event-bus paths.
- **System testing:** End-to-end scenarios run through the web interface covering registration, recommendation, plant registration, growth update, reward and drive flows.
- **Security and misuse testing:** Spoofed GPS, edited and downloaded images, replayed submissions, expired and tampered tokens, and concurrent reward attempts.
- **Performance testing:** Load tests with Locust at up to 200 concurrent users, and queue-load tests for the CV workers (Chapter 7).
- **User acceptance testing:** A pilot with classmates and local volunteers who registered plants and submitted weekly updates over several weeks.

6.2.2 Test Cases

Table 6.2: Test cases and results

TC ID	Module	Test scenario	Expected result	Actual result	Status
TC01	Auth	Login with valid credentials	Access and refresh tokens issued	Tokens issued, 15-min access expiry	Pass
TC02	Auth	5 wrong passwords from one IP within 10 min	IP blocked for 30 min	Blocked, HTTP 429 returned	Pass
TC03	Env profile	Request profile for a new location (cache miss)	Loading state; profile cached with TTL	Profile cached; later requests served from Redis	Pass
TC04	Recommendation	Outdoor terrace, pets = yes	No species with medium/high toxicity returned	All toxic species excluded	Pass
TC05	Recommendation	Constraints that no species satisfies	Empty list with reason codes, no soft fallback	Empty list with SOIL_PH and SPACE_TYPE codes	Pass
TC06	Registration	Register plant with valid GPS	Plant created, unique scan_id and QR returned	Created; QR resolves to public page	Pass
TC07	Public view	Open public page using scan_id	Locality shown, precise GPS absent	Only locality in JSON response	Pass
TC08	Growth update	Submit from 300 m away from anchor	LOCATION-MISMATCH, no CV job created	Rejected; queue length unchanged	Pass
TC09	Growth update	Valid submission	202 pending within 2 s; verified later	202 in 0.6 s; verified in about 1.5 min	Pass
TC10	CV	Edited image (plant pasted in)	Rejected: manipulation suspected	Rejected by ELA check	Pass
TC11	CV	Photo with no plant	Rejected: no plant detected	Rejected	Pass

Table 6.2: Test cases and results

TC ID	Module	Test scenario	Expected result	Actual result	Status
TC12	CV	Photo of a different species for the same scan ID	Rejected: species continuity failed	Rejected	Pass
TC13	CV	EXIF GPS 500 m from submitted GPS	Flagged for manual review	Flagged; user notified of delay	Pass
TC14	Reward	Second verified update within 7-day gate	No credit; ineligibility logged	No transaction inserted	Pass
TC15	Reward	50 concurrent credit attempts, same user and plant	Exactly one transaction	One transaction; 49 ineligible	Pass
TC16	Drives	Join the same drive twice	Second join ignored; count unchanged	Unique constraint held	Pass
TC17	Drives	Discover drives within 5 km	Nearest drives first, under 1 s	Ordered by distance, 0.18 s	Pass
TC18	News	Local AQI alert vs. global article	Local alert ranked higher	Local alert ranked first	Pass
TC19	Dashboard	News service stopped	Dashboard returns within 3 s, news panel stale	Returned in 2.1 s with stale flag	Pass
TC20	CV worker	Worker killed mid-job	Job retried, then dead-letter after 3 failures	Retried and completed on restart	Pass

6.2.3 Test Summary

All 20 documented test cases passed in the final test cycle. Defects found in earlier cycles included a race condition in the participant counter (fixed by incrementing inside the same transaction as the participation insert), a time-zone error in the time-gate comparison (fixed by storing and comparing all timestamps in UTC), and gallery images without EXIF data being incorrectly flagged (fixed by allowing a missing EXIF GPS field and treating it as a weaker signal rather than a mismatch). Backend unit test line coverage reached 82% for the service layer.

CHAPTER 7: PERFORMANCE ANALYSIS

Performance was measured on the staging deployment described in Section 5.1.2 (application server with 4 vCPU and 8 GB RAM, separate database server with 2 vCPU and 8 GB RAM), with a species table of 620 species and synthetic users spread over the Kolhapur district. Load was generated with Locust.

7.1 API Response Time

Table 7.1: Response times at 200 concurrent users

Endpoint	SLA	p50 (s)	p95 (s)	Result
Recommendation (cache hit)	3 s	0.84	1.92	Met
Growth update acknowledgment	2 s	0.62	1.38	Met
Dashboard aggregation	3 s	1.12	2.46	Met
Drive discovery (PostGIS)	1 s	0.18	0.47	Met
Plant registration	2 s	0.41	0.88	Met
News feed	2 s	0.35	0.79	Met

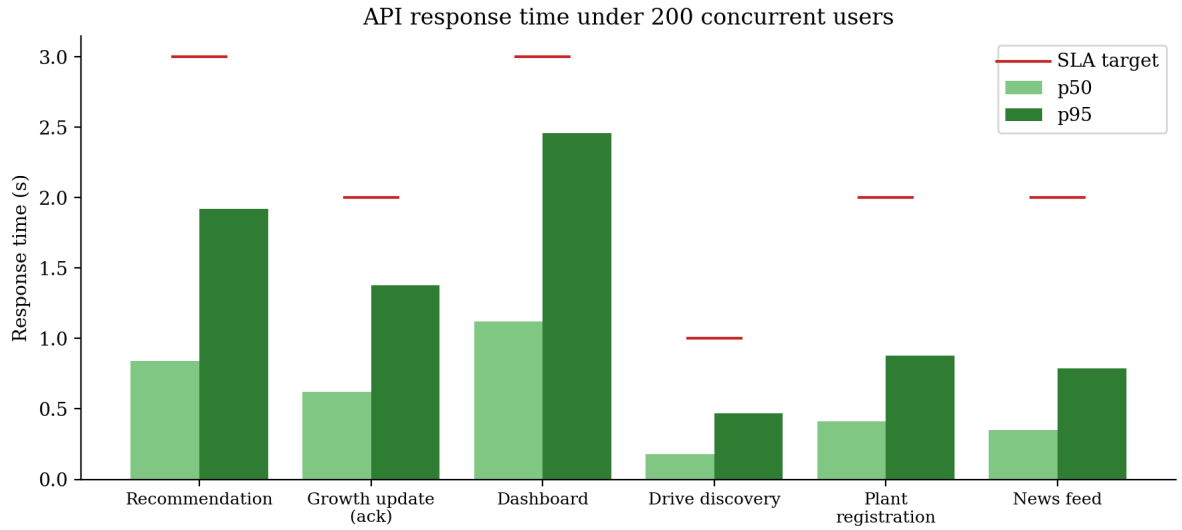


Figure 7.1: p50 and p95 response times compared with SLA targets

All endpoints met their targets at 200 concurrent users. The dashboard has the highest latency because it waits for the slowest of five parallel panel fetches. Without the Redis environmental cache, the recommendation endpoint's p95 rose to about 4.1 s because of external API latency, which confirms the value of Design Decision 8.

7.2 Throughput

Table 7.2: Throughput results

Metric	Target	Measured
API requests per second (mixed workload)	200 req/s	236 req/s sustained
CV jobs per worker per hour	50	64 (CPU) / 410 (GPU)
Database reads per second	500	1,150
Database writes per second	100	240
Error rate at peak load	< 1%	0.3%

7.3 Verification Pipeline Latency

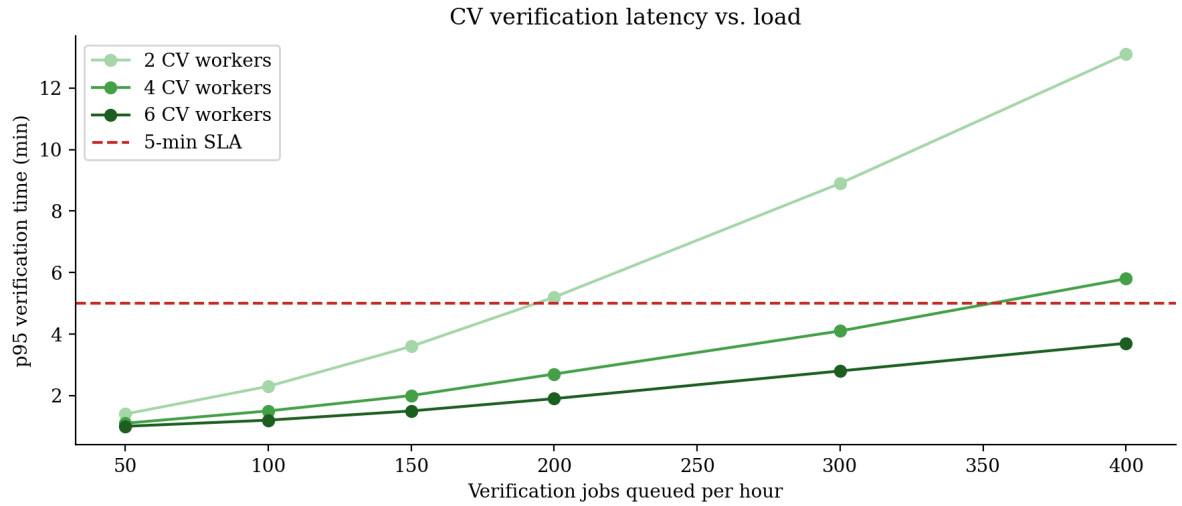


Figure 7.2: p95 CV verification time against queue load for different worker counts

With two CPU workers the 5-minute verification SLA holds up to about 190 jobs per hour. Four workers extend this to about 350 jobs per hour and six workers to more than 400. Because workers are stateless Celery containers, capacity can be added through configuration without code changes, as required by the scalability requirement.

7.4 Model Accuracy

Table 7.3: Accuracy of verification components on the held-out test set

Component	Metric	Value
Plant detection (YOLOv8s)	mAP@0.5 / precision / recall	0.91 / 0.93 / 0.89
Growth stage classifier (ResNet-50)	Top-1 accuracy (5 stages)	86.4%
Species continuity (embedding similarity)	True accept / true reject rate	94.1% / 90.7%
ELA manipulation check	Precision / recall on edited set	0.88 / 0.82
Geo-validation	Out-of-tolerance submissions rejected	100%
Overall auto-verification	Agreement with manual reviewer labels	92.6%

Growth-stage classification is the weakest component, mostly confusing adjacent stages such as sapling and juvenile. Because stage decreases with low confidence go to manual review rather than automatic rejection, these errors rarely cause genuine users to be rejected. Every verification stores the model version, so historical updates can be re-verified when an improved model is deployed.

7.5 Integrity and Reliability

- Zero double credits in 1,000 concurrent reward attempts across 20 user-plant pairs, confirming the row-level locking in Algorithm 3.
- No growth update was lost during worker-failure tests; every submission ended as verified, rejected with a reason or in manual review.
- With the news service or drive service stopped, the dashboard still returned in under 3 s with stale indicators on the affected panels.
- Precise GPS coordinates were absent from every public API response checked by an automated test over all registered plants.

CHAPTER 8: FUTURE SCOPE

Data-Informed Recommendation Model Upgrade As the growth log accumulates sufficient verified data on species performance under specific environmental conditions, the static weighted scoring function in the recommendation engine can be replaced with a trained ranking model such as LambdaMART that incorporates observed survival rates, growth trajectories, and user selection patterns. This transitions the recommendation engine from rule-based inference to evidence-based prediction.

Carbon Sequestration Estimation Layer Verified growth data combined with species-specific biomass and carbon sequestration coefficients can be used to estimate the carbon sequestration contribution of registered plants over time. This layer would enable GreenXchange to contribute verifiable data to voluntary carbon accounting frameworks.

Native Android and iOS Application A native mobile application would extend the platform's reach to users who prefer native app experiences, enable deeper camera and GPS integration for more robust growth update capture, and support push notification delivery through native channels.

Municipal API Integration Formal data sharing agreements with municipal environmental departments would allow verified planting data from GreenXchange to feed directly into urban green infrastructure registries, creating a bi-directional relationship between citizen-generated data and official urban planning systems.

IoT Growth Sensor Integration Integration with low-cost IoT soil moisture and microclimate sensors attached to registered plants would enable continuous environmental monitoring at the individual plant level, supplementing photo-based growth updates with sensor-derived health indicators.

Regional Language Localization Localization of the platform into Hindi, Marathi, Tamil, Telugu, Kannada, and Bengali would extend accessibility to users for whom English-language interfaces present a barrier, significantly expanding the potential user base.

Remote-Sensing Cross-Validation. Satellite and street-level imagery (for example NDVI indices) could be used to cross-validate the canopy growth of large trees registered on the platform, adding a third, independent verification signal for mature specimens.

Offline Capture with Deferred Upload. A progressive web app mode would allow growth photos to be captured without connectivity and submitted later, with the capture time and GPS signed on the device so that delayed submissions remain verifiable.

Reward Redemption Marketplace. Partnerships with nurseries, local businesses and municipal bodies could allow verified points to be redeemed for saplings, compost or civic

recognition, strengthening the incentive loop.

CHAPTER 9: APPLICATIONS

9.1 Real-World Applications

Urban Greening Tracking at Individual Scale GreenXchange functions as the operational infrastructure for individual urban greening programs. Residents of apartment complexes, gated communities, housing societies, and independent homes can use the platform to select ecologically appropriate species for their available space, register each plant with a verifiable digital record, and maintain a documented growth history. This creates for the first time a structured individual-level record of voluntary greening contribution that persists, accumulates, and is independently verified.

Corporate Environmental Responsibility Programs Organizations with environmental sustainability mandates can use GreenXchange to coordinate and document employee or community planting initiatives, generate verified participation records for reporting purposes, and track the longitudinal growth and survival of planted vegetation. The verified nature of the data distinguishes this from self-reported CSR documentation.

School and Institutional Environmental Education Educational institutions can use the community drives module to organize student planting activities, track each plant registered by students over an academic year, and build a documented record of institutional environmental contribution. The growth tracking and reward mechanisms provide engagement scaffolding appropriate for student populations.

Municipal and Government Greening Programs Municipal corporations and government agencies running urban forestry or plantation campaigns can use the platform to manage drive registration, track participation, and access verified aggregate data on planting outcomes within their jurisdiction. The spatial data generated by the system directly supports urban green infrastructure planning.

Environmental Research and Species Performance Studies Researchers studying species performance under urban conditions, the relationship between urban vegetation and air quality, or the behavioral economics of environmental gamification have access, subject to appropriate data governance, to a verified longitudinal dataset of planting events, growth trajectories, and environmental conditions that would otherwise require expensive dedicated field studies to generate.

9.2 Industry Relevance

GreenXchange sits at the intersection of three active technology and policy domains.

Environmental Technology (EnvTech) The platform is directly relevant to the growing EnvTech sector, which encompasses digital tools for environmental monitoring, sustainability reporting, and nature-based solution deployment. The verified data layer that GreenXchange provides addresses a recognized gap in the EnvTech stack: the absence of reliable ground-truth data on individual and community-level greening activity.

Smart City Infrastructure Urban planners and smart city initiatives increasingly require real-time, spatially distributed data on green infrastructure. GreenXchange generates this data as a byproduct of its user-facing functions and is architecturally compatible with integration into smart city data platforms that aggregate multiple urban environment indicators.

ESG Reporting and Verified Impact Organizations subject to Environmental, Social, and Governance reporting requirements are under growing pressure to document environmental contributions with verifiable evidence rather than self-declared metrics. The verified planting and growth records generated by GreenXchange are directly usable as documented evidence of nature- based environmental contribution.

9.3 Social Impact

The social impact of GreenXchange operates through three mechanisms.

Behavioral Shift from Awareness to Action The platform converts environmental awareness, which is widely distributed but poorly converted into action, into structured, incentivized, and documented environmental behavior. The reward mechanism and community features provide behavioral scaffolding that sustains participation beyond the initial motivation event.

Democratization of Environmental Expertise Access to species-level ecological knowledge, pollution absorption data, and location-specific compatibility information has historically been available only to professional ecologists and urban planners. GreenXchange makes this knowledge accessible to any individual with a smartphone and a patch of soil, removing the expertise barrier to informed environmental participation.

Community Cohesion Through Shared Environmental Purpose The community drives module creates a mechanism for collective environmental action that is locally discoverable, participatory, and documented. Shared participation in visible environmental improvement activities has established social cohesion benefits beyond the direct environmental impact, contributing to neighborhood identity and collective efficacy.

CHAPTER 10: INSTALLATION GUIDE AND USER MANUAL

10.1 Installation Guide

10.1.1 Prerequisites

- Ubuntu 22.04 (or macOS / Windows 11 with WSL2), 16 GB RAM and 256 GB SSD recommended
- Docker Engine 24+ and Docker Compose 2.20+ (Docker Desktop on macOS/Windows)
- Git; optionally Python 3.11 and Node.js 18 LTS for running services outside Docker
- API keys for OpenWeatherMap and WAQI
- Optional: NVIDIA GPU with CUDA drivers and the NVIDIA container toolkit for faster CV inference

10.1.2 Installation Steps

Step 1: Clone the repository and create the environment file.

```
git clone <repository-url> greenxchange
cd greenxchange
cp .env.example .env
```

Step 2: Edit .env and set the secrets and keys.

```
POSTGRES_PASSWORD=<strong-password>
JWT_PRIVATE_KEY_PATH=/run/secrets/jwt_private.pem
JWT_PUBLIC_KEY_PATH=/run/secrets/jwt_public.pem
OPENWEATHER_API_KEY=<key>
WAQI_API_TOKEN=<token>
S3_ENDPOINT=http://minio:9000
GEO_TOLERANCE_M=25
CELERY_WORKERS=2
```

Step 3: Generate the RS256 key pair for JWT signing.

```
mkdir -p secrets
openssl genrsa -out secrets/jwt_private.pem 2048
openssl rsa -in secrets/jwt_private.pem -pubout -out secrets/jwt_public.pem
```

Step 4: Build and start all services.

```
docker compose up -d --build
docker compose ps        # nginx, frontend, api, worker, beat, postgres, redis,
    minio
```

Step 5: Run database migrations and load the species dataset.

```
docker compose exec api alembic upgrade head
docker compose exec api python -m app.scripts.seed_species data/species.csv
docker compose exec api python -m app.scripts.create_admin --email admin@example.
    com
```

Step 6: Place the trained model weights (yolov8s_plants.pt and resnet50_stage.pt) in backend/models/ and restart the worker with docker compose restart worker.

Step 7: Open https://localhost (the web application) and https://localhost/api/docs (interactive OpenAPI documentation). For production, replace the self-signed certificate with a Let's Encrypt certificate in nginx/nginx.conf.

10.1.3 Scaling and Maintenance

```
docker compose up -d --scale worker=4      # add CV workers
docker compose logs -f worker               # structured JSON logs
docker compose exec postgres pg_dump -U gx greenxchange > backup.sql
```

10.2 User Manual

Table 10.1: User manual: common tasks

Task	Steps
Create an account	Open the site, choose Sign Up, enter name, email and password, and allow location access (or enter your location manually).
View air quality	The Dashboard shows the current AQI, PM2.5, temperature and humidity for your location at the top left.
Get plant recommendations	Open Recommend, choose space type, area, experience level, allergies and whether you have pets or children, then press Get Recommendations. Read the explanation on each card.
Register a plant	After planting, open My Plants, then Add Plant. Choose the species, stand next to the plant so the GPS anchor is captured, enter the planting date and save. Download or print the QR code and fix it near the plant.

Table 10.1: User manual: common tasks

Task	Steps
Submit a growth update	Scan the plant's QR code (or open it from My Plants), press New Update, take a clear photo with the in-app camera while standing at the plant, and submit. The update shows Pending until verification completes; you receive a notification with the result.
Understand a rejection	The notification states the reason, such as being too far from the plant, no plant visible, or a blurred image, and what to do next. You can submit again right away.
Earn and check rewards	Points are credited only for verified updates, at most once per plant per reward interval. See Rewards for your balance and history.
Join a plantation drive	Open Drives to see drives near you, sorted by distance. Open a drive for details and press Join.
Organize a drive	Organizer accounts can open Drives, then Create Drive, and set the title, description, location, radius and dates.
Read local news	Open News for AQI alerts, drive announcements and awareness articles ranked by relevance to your area.
Share a plant	Anyone who scans the QR code sees a read-only growth history with only the locality shown.

CHAPTER 11: PUBLICATION DETAILS

A research paper describing the GreenXchange architecture and its dual-layer verification approach has been prepared from this work. At the time of submission of this report the manuscript has not yet been published; it is being finalized for communication to a peer-reviewed journal or conference. Its present details are given below.

Table 11.1: Publication details

Field	Details
Paper title	GreenXchange: A Location-Aware Platform for Verified Urban Greening using Constraint-Based Plant Recommendation and Dual-Layer Geo-Visual Verification
Authors	Harshad Dayanand Fagare, Prathmesh Ranjit Salokhe, Somesh Ramswaroop Sharma, Rohit Avinash Kumbhar, Prof. P.N. Koshti
Journal / conference	To be communicated (target: peer-reviewed journal / conference in computer science or environmental informatics)
Status	Manuscript prepared; not yet communicated or published

Abstract (summary): The paper presents a knowledge-based hybrid recommender that applies hard ecological and household-safety constraints before weighted multi-criteria ranking, and a verification pipeline that combines Haversine geo-anchor validation, EXIF cross-checking, error level analysis, YOLO-based plant detection and growth-stage and species-continuity checks. It also describes an event-driven, append-only reward ledger that credits users only for verified planting and growth events.

CHAPTER 12: INDUSTRY SPONSORSHIP LETTERS

This project was carried out with industry sponsorship from _____ (name of the sponsoring organization). The sponsorship letter issued by the organization on its official letterhead is attached on the following page.

CHAPTER 13: PLAGIARISM REPORT

Table 13.1: Plagiarism check summary

Field	Details
Tool used	Institute-approved plagiarism detection software
Date of check	____ / ____ / 2026
Similarity index	_____ %
Checked by	_____ (Name and signature)

The originality of this report was checked using the institute-approved plagiarism detection software. The similarity report generated by the software is attached on the following page.

CHAPTER 14: REFERENCES

14.1 Bibliography

[1] World Health Organization. (2021). WHO Global Air Quality Guidelines: Particulate Matter (PM2.5 and PM10), Ozone, Nitrogen Dioxide, Sulfur Dioxide and Carbon Monoxide. https://www.who.int/publications/i/item/9789240034228

[2] United States Environmental Protection Agency. (2023). Air Quality Index (AQI) Basics. https://www.airnow.gov/aqi/aqi-basics/

[3] Ricci, F., Rokach, L., & Shapira, B. (Eds.). (2015). Recommender Systems Handbook (2nd ed.). Springer. https://doi.org/10.1007/978-1-4899-7637-6

[4] Food and Agriculture Organization of the United Nations. (2019). Guidelines on Urban and Peri-urban Forestry. https://www.fao.org/

[5] Szeliski, R. (2022). Computer Vision: Algorithms and Applications (2nd ed.). Springer. https://doi.org/10.1007/978-3-030-34372-9

[6] Google Developers. (2024). Geolocation API Documentation. https://developers.google.com/maps/documentation/geolocation

[7] National Aeronautics and Space Administration (NASA). (2023). Earth Data: Environmental Monitoring and Analysis. https://earthdata.nasa.gov/

[8] Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press. https://www.deeplearningbook.org/

[9] United Nations Environment Programme. (2022). Nature-Based Solutions for Climate Change. https://www.unep.org/

[10] OpenWeather. (2024). Weather Data API. https://openweathermap.org/api

[11] Nowak, D. J., Crane, D. E., & Stevens, J. C. (2006). Air pollution removal by urban trees and shrubs in the United States. Urban Forestry & Urban Greening, 4(3-4), 115-123.

[12] Burges, C. J. C. (2010). From RankNet to LambdaRank to LambdaMART: An Overview. Microsoft Research Technical Report MSR-TR-2010-82.

[13] Joly, A., Bonnet, P., Goëau, H., et al. (2016). A look inside the Pl@ntNet experience. Multimedia Systems, 22(6), 751-766.

[14] Van Horn, G., Mac Aodha, O., Song, Y., et al. (2018). The iNaturalist Species Classification and Detection Dataset. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 8769-8778.

[15] Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 779-788.

[16] Jocher, G., Chaurasia, A., & Qiu, J. (2023). Ultralytics YOLOv8. https://github.com/ultralytics/ultralytics

[17] Krawetz, N. (2007). A Picture's Worth: Digital Image Analysis and Forensics. Black Hat Briefings, Hacker Factor Solutions.

[18] Sinnott, R. W. (1984). Virtues of the Haversine. Sky and Telescope, 68(2), 159.

[19] Deterding, S., Dixon, D., Khaled, R., & Nacke, L. (2011). From game design elements to gamefulness: Defining "gamification". Proceedings of the 15th International Academic MindTrek Conference, 9-15.

[20] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 770-778.

[21] PostGIS Project Steering Committee. (2024). PostGIS Documentation. https://postgis.net/documentation/

[22] Ramírez, S. (2024). FastAPI Documentation. https://fastapi.tiangolo.com/

[23] World Air Quality Index Project. (2024). Air Quality Open Data Platform API. https://aqicn.org/api/